PawHome — SQL Query Reference

Pet Adoption & Boarding Management System · CSE 3522 Database Management Systems Laboratory · United International University

Queries documented	**166**
Java files they live in	**41**
Reached by an HTTP endpoint	**145**
Run by a scheduled background job	**7**
Present but not called by anything	**14**
REST endpoints in the backend	**151**
Base tables / views in the schema	**32 / 4**

How to read this

file name — the Java file the query is written in, relative to `backend/src/main/java/com/pawhome/backend/`.
api name — the HTTP method and path of every endpoint that runs it. Several endpoints often share one query: the admin and staff versions of a screen usually call the same service method with a different scope.
Query — the SQL as it actually executes. Where the code builds a statement by concatenating a shared constant with a WHERE clause, the constant has been substituted in, so what is printed is the whole statement rather than the fragment in the source.

*Two dialects appear. Queries in a **…Repository.java** file are **JPQL** — Spring Data JPA's own language, which names Java entities and fields (`BoardingRequest b … b.petId`) and is translated to SQL by Hibernate at runtime. Queries in a **…Service.java** file are **raw SQL** run through JdbcTemplate and name real tables and columns. Only the second kind can be pasted straight into the mysql client.*

admin/AdminUserService.java — 4 queries

file name: `admin/AdminUserService.java`
api name: GET /api/v1/admin/users/counts
Query: (SQL · SELECT · counts)

```
SELECT  COUNT(*)                                              AS total,
        SUM(role = 'STAFF')                                   AS staff,
        SUM(role = 'ADOPTER')                                 AS adopters,
        SUM(role = 'STAFF' AND approval_status = 'PENDING')   AS awaitingApproval,
        SUM(email_verified = FALSE)                           AS unverified
FROM users
```

file name: `admin/AdminUserService.java`
api name: GET /api/v1/admin/users
Query: (SQL · SELECT · list)

```
SELECT  u.id, u.full_name, u.email, u.role, u.approval_status,
        u.email_verified, u.is_active, u.shelter_id,
        s.name AS shelter_name, u.job_title, u.created_at,
        -- An outstanding invitation. Correlated EXISTS rather than
        -- a join, because a join to user_tokens would duplicate the
        -- user row once per token they have ever been sent.
        EXISTS (SELECT 1
                  FROM user_tokens t
                 WHERE t.user_id     = u.id
                   AND t.token_type  = 'INVITE'
                   AND t.used_at     IS NULL
                   AND t.expires_at  > UTC_TIMESTAMP()) AS invite_pending
FROM        users    u
LEFT JOIN   shelters s ON s.id = u.shelter_id
ORDER BY    u.created_at DESC
```

file name: `admin/AdminUserService.java`
api name: POST /api/v1/admin/users/{id}/activate
POST /api/v1/admin/users/{id}/approve
POST /api/v1/admin/users/{id}/deactivate
POST /api/v1/admin/users/{id}/reject
POST /api/v1/admin/users/{id}/shelter
POST /api/v1/admin/users/{id}/verify-email
Query: (SQL · SELECT · row)

```
SELECT name FROM shelters WHERE id = ?
```

file name: `admin/AdminUserService.java`
api name: POST /api/v1/admin/users/{id}/activate
POST /api/v1/admin/users/{id}/approve
POST /api/v1/admin/users/{id}/deactivate
POST /api/v1/admin/users/{id}/reject
POST /api/v1/admin/users/{id}/shelter
POST /api/v1/admin/users/{id}/verify-email
Query: (SQL · SELECT · row)

```
SELECT EXISTS (SELECT 1 FROM user_tokens
                WHERE user_id = ? AND token_type = 'INVITE'
                  AND used_at IS NULL
                  AND expires_at > UTC_TIMESTAMP())
```

analytics/AnalyticsService.java — 18 queries

file name: `analytics/AnalyticsService.java`
api name: (not called by anything — see the dead-query note)
Query: (SQL · SELECT · APPS_IN_WINDOW)

```
SELECT COUNT(*) FROM adoption_applications app
  JOIN pets p ON p.id = app.pet_id
 WHERE app.submitted_at >= ?
   AND (? IS NULL OR p.shelter_id = ?)
```

file name: analytics/AnalyticsService.java
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · activePetTile)

```
SELECT COUNT(*) FROM pets p
 WHERE p.shelter_id IS NOT NULL
   AND p.status IN ('AVAILABLE','PENDING')
   AND (? IS NULL OR p.shelter_id = ?)
```

file name: analytics/AnalyticsService.java
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · adoptionRecords)

```
SELECT a.id, p.name AS pet, sh.name AS shelter, u.full_name AS adopter,
       a.adopted_on AS adopted_on, a.fee_paid,
       -- CAN COME OUT NEGATIVE, AND IS NOT CLAMPED HERE.
       -- Nothing stops an adoption being dated before the
       -- application it came from: a CHECK cannot span two
       -- tables in MySQL, so the ordering is a service-level
       -- invariant at adoption time and not a guarantee this
       -- query may lean on. GREATEST(...,0) would turn a
       -- data-entry error into a plausible "0 days" and hide
       -- it forever; the real figure travels, and the table
       -- renders anything negative as "--".
       DATEDIFF(a.adopted_on, DATE(app.submitted_at)) AS days
  FROM adoptions a
  JOIN pets p                   ON p.id   = a.pet_id
  JOIN shelters sh              ON sh.id  = a.shelter_id
  JOIN users u                  ON u.id   = a.adopter_id
  JOIN adoption_applications app ON app.id = a.application_id
 WHERE a.adopted_on >= ?
   AND (? IS NULL OR a.shelter_id = ?)
 ORDER BY a.adopted_on DESC, a.id DESC
 LIMIT 25
```

file name: analytics/AnalyticsService.java
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · adoptionTile)

```
SELECT COUNT(*) FROM adoptions a
 WHERE a.adopted_on >= ?
   AND (? IS NULL OR a.shelter_id = ?)
```

file name: analytics/AnalyticsService.java
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · adoptionTile)

```
SELECT COUNT(*) FROM adoptions a
 WHERE a.adopted_on >= ? AND a.adopted_on < ?
   AND (? IS NULL OR a.shelter_id = ?)
```

file name: analytics/AnalyticsService.java
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · adoptionsByMonth)

```
WITH RECURSIVE months AS (
    SELECT DATE_FORMAT(?, '%Y-%m-01') AS m
    UNION ALL
    SELECT DATE_ADD(m, INTERVAL 1 MONTH) FROM months
     WHERE m < DATE_FORMAT(?, '%Y-%m-01')
)
SELECT DATE_FORMAT(months.m, '%b %Y') AS month,
       COUNT(a.id)                    AS adoptions
  FROM months
  LEFT JOIN adoptions a
         ON a.adopted_on >= months.m
        AND a.adopted_on <  DATE_ADD(months.m, INTERVAL 1 MONTH)
        AND (? IS NULL OR a.shelter_id = ?)
 GROUP BY months.m
 ORDER BY months.m
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: **(SQL · SELECT · adoptionsByShelter)**

```
SELECT sh.name AS shelter, COUNT(a.id) AS adoptions
  FROM shelters sh
  LEFT JOIN adoptions a
         ON a.shelter_id = sh.id AND a.adopted_on >= ?
 GROUP BY sh.id, sh.name
 ORDER BY adoptions DESC, sh.name
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: **(SQL · SELECT · applicationTile)**

```
SELECT COUNT(*) FROM adoption_applications app
  JOIN pets p ON p.id = app.pet_id
 WHERE app.submitted_at >= ? AND app.submitted_at < ?
   AND (? IS NULL OR p.shelter_id = ?)
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: **(SQL · SELECT · applicationsByStatus)**

```
SELECT s.label AS name, COUNT(app.id) AS value
  FROM (          SELECT 'Approved'  AS label, 'APPROVED'  AS code
        UNION ALL SELECT 'Pending',           'PENDING'
        UNION ALL SELECT 'Rejected',          'REJECTED'
        UNION ALL SELECT 'Withdrawn',         'WITHDRAWN') s
  LEFT JOIN (adoption_applications app
             JOIN pets p ON p.id = app.pet_id)
         ON app.status = s.code
        AND app.submitted_at >= ?
        AND (? IS NULL OR p.shelter_id = ?)
 GROUP BY s.label, s.code
 ORDER BY FIELD(s.code, 'APPROVED','PENDING','REJECTED','WITHDRAWN')
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: **(SQL · SELECT · decision)**

```
SELECT SUM(app.status = 'APPROVED')                      AS approved,
       SUM(app.status IN ('APPROVED','REJECTED'))        AS decided
  FROM adoption_applications app
  JOIN pets p ON p.id = app.pet_id
 WHERE app.submitted_at >= ?
AND (? IS NULL OR p.shelter_id = ?)
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: **(SQL · SELECT · decision)**

```
SELECT SUM(app.status = 'APPROVED')                      AS approved,
       SUM(app.status IN ('APPROVED','REJECTED'))        AS decided
  FROM adoption_applications app
  JOIN pets p ON p.id = app.pet_id
 WHERE app.submitted_at >= ?
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: **(SQL · SELECT · nightsTile)**

```
SELECT COALESCE(SUM(nights_billed), 0)
  FROM v_shelter_revenue
 WHERE billed_month >= ?
   AND (? IS NULL OR shelter_id = ?)
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · overview)

```
SELECT COUNT(*) FROM shelters WHERE id = ?
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · petsBySpecies)

```
SELECT p.species,
       SUM(p.status = 'AVAILABLE')   AS available,
       SUM(p.status = 'PENDING')     AS pending,
       SUM(p.status = 'ADOPTED')     AS adopted,
       SUM(p.status = 'UNAVAILABLE') AS unavailable
  FROM pets p
 WHERE p.shelter_id IS NOT NULL
   AND (? IS NULL OR p.shelter_id = ?)
 GROUP BY p.species
 ORDER BY p.species
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · revenueByMonth)

```
WITH RECURSIVE months AS (
    SELECT DATE_FORMAT(?, '%Y-%m-01') AS m
    UNION ALL
    SELECT DATE_ADD(m, INTERVAL 1 MONTH) FROM months
     WHERE m < DATE_FORMAT(?, '%Y-%m-01')
)
SELECT DATE_FORMAT(months.m, '%b %Y')                  AS month,
       COALESCE(SUM(r.boarding_slips), 0)              AS boarding_slips,
       COALESCE(SUM(r.nights_billed), 0)               AS nights_billed,
       COALESCE(SUM(r.boarding_total), 0.00)           AS boarding_total,
       COALESCE(SUM(r.daycare_slips), 0)               AS daycare_slips,
       COALESCE(SUM(r.months_billed), 0)               AS months_billed,
       COALESCE(SUM(r.daycare_total), 0.00)            AS daycare_total,
       COALESCE(SUM(r.total_billed), 0.00)             AS total_billed
  FROM months
  LEFT JOIN v_shelter_revenue r
         ON r.billed_month = months.m
        AND (? IS NULL OR r.shelter_id = ?)
 GROUP BY months.m
 ORDER BY months.m
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · revenueByShelter)

```
SELECT  s.id                                  AS shelter_id,
        s.name                                AS shelter,
        COALESCE(SUM(r.nights_billed), 0)     AS nights_billed,
        COALESCE(SUM(r.boarding_total), 0.00) AS boarding_total,
        COALESCE(SUM(r.daycare_total), 0.00)  AS daycare_total,
        COALESCE(SUM(r.total_billed), 0.00)   AS total_billed,
        (SELECT COUNT(*) FROM daycare_subscriptions d
          WHERE d.shelter_id = s.id AND d.status = 'ACTIVE')
                                              AS active_subscriptions
FROM        shelters s
LEFT JOIN   v_shelter_revenue r
        ON  r.shelter_id = s.id AND r.billed_month >= ?
GROUP BY    s.id, s.name
ORDER BY    total_billed DESC, s.name
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: (SQL · SELECT · revenueTile)

```
SELECT COALESCE(SUM(%s), 0)
  FROM v_shelter_revenue
 WHERE billed_month >= ?
   AND (? IS NULL OR shelter_id = ?)
```

file name: `analytics/AnalyticsService.java`
api name: GET /api/v1/admin/analytics
Query: **(SQL · SELECT · subscriptionTile)**

```
SELECT COUNT(*) FROM daycare_subscriptions
 WHERE status = 'ACTIVE'
   AND (? IS NULL OR shelter_id = ?)
```

application/AdoptionRepository.java — 1 query

file name: `application/AdoptionRepository.java`
api name: (not called by anything — see the dead-query note)
Query: **(JPQL · SELECT · findByApplicationId)**

```
SELECT *
  FROM adoptions
 WHERE application_id = :applicationId
```

application/ApplicationRepository.java — 6 queries

file name: `application/ApplicationRepository.java`
api name: POST /api/v1/staff/applications/{id}/approve
Query: **(JPQL · UPDATE · closeRivalApplications)**

```
UPDATE adoption_applications
   SET status              = 'REJECTED',
       reviewed_by         = :reviewerId,
       reviewed_at         = :decidedAt,
       decline_reason_code = 'PET_ALREADY_MATCHED',
       decline_message     = :message
 WHERE pet_id = :petId
   AND id    <> :keepId
   AND status = 'PENDING'
```

file name: `application/ApplicationRepository.java`
api name: POST /api/v1/applications
Query: **(JPQL · SELECT · existsByApplicantIdAndPetIdAndStatus)**

```
SELECT COUNT(a) > 0
  FROM AdoptionApplication a
 WHERE a.applicantId = :applicantId
   AND a.petId = :petId
   AND a.status = :status
```

file name: `application/ApplicationRepository.java`
api name: POST /api/v1/staff/applications/{id}/approve
Query: **(JPQL · SELECT · findByPetIdAndStatus)**

```
SELECT *
  FROM adoption_applications
 WHERE pet_id = :petId
   AND status = :#{#status.name()}
```

file name: `application/ApplicationRepository.java`
api name: POST /api/v1/staff/applications/{id}/approve
Query: **(JPQL · UPDATE · markApproved)**

```
UPDATE adoption_applications
   SET status      = 'APPROVED',
       reviewed_by = :reviewerId,
       reviewed_at = :decidedAt
 WHERE id     = :id
   AND status = 'PENDING'
```

file name: `application/ApplicationRepository.java`
api name: POST /api/v1/staff/applications/{id}/decline
Query: (JPQL · UPDATE · markDeclined)

```
UPDATE adoption_applications
   SET status              = 'REJECTED',
       reviewed_by         = :reviewerId,
       reviewed_at         = :decidedAt,
       decline_reason_code = :#{#reasonCode.name()},
       decline_message     = :message
 WHERE id     = :id
   AND status = 'PENDING'
```

file name: `application/ApplicationRepository.java`
api name: POST /api/v1/applications/{id}/withdraw
Query: (JPQL · UPDATE · markWithdrawn)

```
UPDATE adoption_applications
   SET status      = 'WITHDRAWN',
       reviewed_at = :withdrawnAt
 WHERE id     = :id
   AND status = 'PENDING'
```

application/ApplicationService.java — 4 queries

file name: `application/ApplicationService.java`
api name: GET /api/v1/staff/applications/counts
Query: (SQL · SELECT · counts)

```
SELECT  COUNT(*)                     AS total,
        SUM(a.status = 'PENDING')    AS pending,
        SUM(a.status = 'APPROVED')   AS approved,
        SUM(a.status = 'REJECTED')   AS rejected,
        SUM(a.status = 'WITHDRAWN')  AS withdrawn
FROM       adoption_applications a
JOIN       pets p ON p.id = a.pet_id
WHERE      (? IS NULL OR p.shelter_id = ?)
```

file name: `application/ApplicationService.java`
api name: GET /api/v1/staff/applications
Query: (SQL · SELECT · forShelter)

```
SELECT  a.id, a.pet_id, p.name AS pet_name, p.breed AS pet_breed, p.status AS pet_status,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        p.shelter_id, s.name AS shelter_name,
        a.applicant_id, a.applicant_name, a.applicant_email, a.applicant_phone,
        a.applicant_address, a.applicant_city, a.home_type, a.time_at_home,
        a.pet_experience, a.household_note, a.quiz_score,
        q.max_score AS quiz_max_score, q.passed AS quiz_passed,
        a.status, a.submitted_at, r.full_name AS reviewed_by_name, a.reviewed_at,
        a.decline_reason_code, a.decline_message,
        ad.adopted_on, ad.fee_paid
FROM        adoption_applications a
JOIN        pets          p ON p.id = a.pet_id
JOIN        shelters      s ON s.id = p.shelter_id
JOIN        quiz_attempts q ON q.id = a.eligibility_attempt_id
LEFT JOIN   users         r ON r.id = a.reviewed_by
-- Cannot fan the result out: uq_adoption_application is UNIQUE on
-- application_id, so this LEFT JOIN matches at most one row.
LEFT JOIN   adoptions    ad ON ad.application_id = a.id
WHERE (? IS NULL OR p.shelter_id = ?)
ORDER BY FIELD(a.status,'PENDING','APPROVED','REJECTED','WITHDRAWN'),
         a.submitted_at
```

file name: `application/ApplicationService.java`
api name: GET /api/v1/applications/mine
Query: (SQL · SELECT · mine)

```
SELECT  a.id, a.pet_id, p.name AS pet_name, p.breed AS pet_breed, p.status AS pet_status,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        p.shelter_id, s.name AS shelter_name,
        a.applicant_id, a.applicant_name, a.applicant_email, a.applicant_phone,
        a.applicant_address, a.applicant_city, a.home_type, a.time_at_home,
        a.pet_experience, a.household_note, a.quiz_score,
        q.max_score AS quiz_max_score, q.passed AS quiz_passed,
        a.status, a.submitted_at, r.full_name AS reviewed_by_name, a.reviewed_at,
        a.decline_reason_code, a.decline_message,
        ad.adopted_on, ad.fee_paid
FROM        adoption_applications a
JOIN        pets          p ON p.id = a.pet_id
JOIN        shelters      s ON s.id = p.shelter_id
JOIN        quiz_attempts q ON q.id = a.eligibility_attempt_id
LEFT JOIN   users         r ON r.id = a.reviewed_by
-- Cannot fan the result out: uq_adoption_application is UNIQUE on
-- application_id, so this LEFT JOIN matches at most one row.
LEFT JOIN   adoptions    ad ON ad.application_id = a.id
 WHERE a.applicant_id = ? ORDER BY a.submitted_at DESC
```

file name: `application/ApplicationService.java`
api name: GET /api/v1/staff/applications/{id}
POST /api/v1/applications
POST /api/v1/applications/{id}/withdraw
POST /api/v1/staff/applications/{id}/approve
POST /api/v1/staff/applications/{id}/decline
Query: (SQL · SELECT · row)

```
SELECT  a.id, a.pet_id, p.name AS pet_name, p.breed AS pet_breed, p.status AS pet_status,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        p.shelter_id, s.name AS shelter_name,
        a.applicant_id, a.applicant_name, a.applicant_email, a.applicant_phone,
        a.applicant_address, a.applicant_city, a.home_type, a.time_at_home,
        a.pet_experience, a.household_note, a.quiz_score,
        q.max_score AS quiz_max_score, q.passed AS quiz_passed,
        a.status, a.submitted_at, r.full_name AS reviewed_by_name, a.reviewed_at,
        a.decline_reason_code, a.decline_message,
        ad.adopted_on, ad.fee_paid
FROM        adoption_applications a
JOIN        pets          p ON p.id = a.pet_id
JOIN        shelters      s ON s.id = p.shelter_id
JOIN        quiz_attempts q ON q.id = a.eligibility_attempt_id
LEFT JOIN   users         r ON r.id = a.reviewed_by
-- Cannot fan the result out: uq_adoption_application is UNIQUE on
-- application_id, so this LEFT JOIN matches at most one row.
LEFT JOIN   adoptions    ad ON ad.application_id = a.id
 WHERE a.id = ?
```

audit/AuditLogRepository.java — 1 query

file name: `audit/AuditLogRepository.java`
api name: (not called by anything — see the dead-query note)
Query: (JPQL · SELECT · findRecent)

```
SELECT *
  FROM audit_log
 ORDER BY created_at DESC, id DESC
 LIMIT :limit
```

audit/AuditService.java — 1 query

file name: `audit/AuditService.java`
api name: GET /api/v1/admin/activity
Query: (SQL · SELECT · recent)

```
SELECT  a.id, a.action, a.entity_type, a.entity_id, a.detail,
        a.created_at,
        COALESCE(u.full_name, 'System') AS actor_name
FROM        audit_log a
LEFT JOIN   users     u ON u.id = a.actor_id
ORDER BY a.created_at DESC, a.id DESC
LIMIT ?
```

billing/BillingService.java — 4 queries

file name: `billing/BillingService.java`
api name: GET /api/v1/admin/slips
GET /api/v1/staff/slips
Query: (SQL · SELECT · forShelter)

```
SELECT  bs.id, bs.slip_no, bs.owner_id, o.full_name AS owner_name,
        o.email AS owner_email,
        bs.shelter_id, s.name AS shelter_name,
        s.address_line AS shelter_address, s.phone AS shelter_phone,
        bs.boarding_request_id, bs.subscription_id,
        d.slot AS slot,
        -- The pet is reached through whichever source this slip has.
        -- COALESCE rather than two columns: ck_slip_source guarantees
        -- exactly one of the two joins produced a row.
        COALESCE(br.pet_id, d.pet_id) AS pet_id,
        p.name AS pet_name,
        bs.period_from, bs.period_to, bs.units, bs.unit_rate, bs.total,
        bs.issued_at, iss.full_name AS issued_by_name,
        bs.units * bs.unit_rate AS base_total,
        -- COALESCE: SUM over nought lines is NULL, and a slip with no
        -- extras is the normal case, not an edge one.
        COALESCE((SELECT SUM(li.line_total) FROM billing_slip_items li
                  WHERE li.slip_id = bs.id), 0)        AS extras_total,
        (SELECT COUNT(*) FROM billing_slip_items li2
          WHERE li2.slip_id = bs.id)                   AS item_count
FROM        billing_slips         bs
JOIN        users                 o   ON o.id   = bs.owner_id
JOIN        shelters              s   ON s.id   = bs.shelter_id
-- Both LEFT, because exactly one of them matches on any given row.
LEFT JOIN   boarding_requests     br  ON br.id  = bs.boarding_request_id
LEFT JOIN   daycare_subscriptions d   ON d.id   = bs.subscription_id
LEFT JOIN   pets                  p   ON p.id   = COALESCE(br.pet_id, d.pet_id)
-- LEFT, because issued_by is NULL on every slip the monthly job wrote.
LEFT JOIN   users                 iss ON iss.id = bs.issued_by
WHERE (? IS NULL OR bs.shelter_id = ?)
ORDER BY bs.issued_at DESC, bs.id DESC
```

file name: `billing/BillingService.java`
api name: GET /api/v1/admin/slips/{id}
GET /api/v1/slips/{id}
GET /api/v1/staff/slips/{id}
Query: (SQL · SELECT · get)

```
SELECT  bs.id, bs.slip_no, bs.owner_id, o.full_name AS owner_name,
        o.email AS owner_email,
        bs.shelter_id, s.name AS shelter_name,
        s.address_line AS shelter_address, s.phone AS shelter_phone,
        bs.boarding_request_id, bs.subscription_id,
        d.slot AS slot,
        -- The pet is reached through whichever source this slip has.
        -- COALESCE rather than two columns: ck_slip_source guarantees
        -- exactly one of the two joins produced a row.
        COALESCE(br.pet_id, d.pet_id) AS pet_id,
        p.name AS pet_name,
        bs.period_from, bs.period_to, bs.units, bs.unit_rate, bs.total,
        bs.issued_at, iss.full_name AS issued_by_name,
        bs.units * bs.unit_rate AS base_total,
        -- COALESCE: SUM over nought lines is NULL, and a slip with no
        -- extras is the normal case, not an edge one.
        COALESCE((SELECT SUM(li.line_total) FROM billing_slip_items li
                  WHERE li.slip_id = bs.id), 0)        AS extras_total,
        (SELECT COUNT(*) FROM billing_slip_items li2
          WHERE li2.slip_id = bs.id)                   AS item_count
FROM        billing_slips         bs
JOIN        users                 o   ON o.id   = bs.owner_id
JOIN        shelters              s   ON s.id   = bs.shelter_id
-- Both LEFT, because exactly one of them matches on any given row.
LEFT JOIN   boarding_requests     br  ON br.id  = bs.boarding_request_id
LEFT JOIN   daycare_subscriptions d   ON d.id   = bs.subscription_id
LEFT JOIN   pets                  p   ON p.id   = COALESCE(br.pet_id, d.pet_id)
-- LEFT, because issued_by is NULL on every slip the monthly job wrote.
LEFT JOIN   users                 iss ON iss.id = bs.issued_by
 WHERE bs.id = ?
```

file name: `billing/BillingService.java`
api name: GET /api/v1/slips/mine
Query: (SQL · SELECT · mine)

```
SELECT  bs.id, bs.slip_no, bs.owner_id, o.full_name AS owner_name,
        o.email AS owner_email,
        bs.shelter_id, s.name AS shelter_name,
        s.address_line AS shelter_address, s.phone AS shelter_phone,
        bs.boarding_request_id, bs.subscription_id,
        d.slot AS slot,
        -- The pet is reached through whichever source this slip has.
        -- COALESCE rather than two columns: ck_slip_source guarantees
        -- exactly one of the two joins produced a row.
        COALESCE(br.pet_id, d.pet_id) AS pet_id,
        p.name AS pet_name,
        bs.period_from, bs.period_to, bs.units, bs.unit_rate, bs.total,
        bs.issued_at, iss.full_name AS issued_by_name,
        bs.units * bs.unit_rate AS base_total,
        -- COALESCE: SUM over nought lines is NULL, and a slip with no
        -- extras is the normal case, not an edge one.
        COALESCE((SELECT SUM(li.line_total) FROM billing_slip_items li
                  WHERE li.slip_id = bs.id), 0)        AS extras_total,
        (SELECT COUNT(*) FROM billing_slip_items li2
          WHERE li2.slip_id = bs.id)                   AS item_count
FROM        billing_slips         bs
JOIN        users                 o   ON o.id   = bs.owner_id
JOIN        shelters              s   ON s.id   = bs.shelter_id
-- Both LEFT, because exactly one of them matches on any given row.
LEFT JOIN   boarding_requests     br  ON br.id  = bs.boarding_request_id
LEFT JOIN   daycare_subscriptions d   ON d.id   = bs.subscription_id
LEFT JOIN   pets                  p   ON p.id   = COALESCE(br.pet_id, d.pet_id)
-- LEFT, because issued_by is NULL on every slip the monthly job wrote.
LEFT JOIN   users                 iss ON iss.id = bs.issued_by
 WHERE bs.owner_id = ? ORDER BY bs.issued_at DESC, bs.id DESC
```

file name: `billing/BillingService.java`
api name: GET /api/v1/admin/slips/totals
GET /api/v1/staff/slips/totals
Query: (SQL · SELECT · totals)

```
SELECT  COUNT(*)                                AS slips,
        COALESCE(SUM(total), 0)                 AS total,
        -- COALESCE on the two SUMs as well as on the total:
        -- SUM over nought rows is NULL, not 0, and a shelter
        -- that has billed nothing yet is the normal first day.
        COALESCE(SUM(boarding_request_id IS NOT NULL), 0) AS boarding_slips,
        COALESCE(SUM(subscription_id     IS NOT NULL), 0) AS daycare_slips
  FROM  billing_slips
 WHERE  (? IS NULL OR shelter_id = ?)
```

billing/BillingSlipItemRepository.java — 1 query

file name: `billing/BillingSlipItemRepository.java`
api name: GET /api/v1/admin/slips/{id}
GET /api/v1/slips/{id}
GET /api/v1/staff/slips/{id}
Query: (JPQL · SELECT · findForSlip)

```
SELECT *
  FROM billing_slip_items
 WHERE slip_id = :slipId
 ORDER BY sort_order, id
```

billing/BillingSlipRepository.java — 2 queries

file name: `billing/BillingSlipRepository.java`
api name: POST /api/v1/staff/boarding/{id}/complete
Query: (JPQL · SELECT · findForBoarding)

```
SELECT *
  FROM billing_slips
 WHERE boarding_request_id = :boardingRequestId
```

file name: `billing/BillingSlipRepository.java`
api name: Scheduled job — MonthlyBillingJob.billTheMonth -> BillingService.issueForSubscription
Query: (JPQL · SELECT · findForSubscriptionMonth)

```
SELECT *
  FROM billing_slips
 WHERE subscription_id = :subscriptionId
   AND period_from     = :periodFrom
```

boarding/BoardingChargeRepository.java — 1 query

file name: `boarding/BoardingChargeRepository.java`
api name: (not called by anything — see the dead-query note)
Query: (JPQL · SELECT · findForBoarding)

```
SELECT *
  FROM boarding_charges
 WHERE boarding_id = :boardingId
 ORDER BY added_at ASC, id ASC
```

boarding/BoardingInstructionRepository.java — 1 query

file name: `boarding/BoardingInstructionRepository.java`
api name: GET /api/v1/boarding/{id}
GET /api/v1/staff/boarding/{id}
Query: (JPQL · SELECT · findByBoardingIdOrderByIdAsc)

```
SELECT *
  FROM boarding_instructions
 WHERE boarding_id = :boardingId
 ORDER BY id ASC
```

boarding/BoardingRequestRepository.java — 4 queries

file name: `boarding/BoardingRequestRepository.java`
api name: POST /api/v1/staff/boarding/{id}/accept
Query: (JPQL · SELECT · countPlacesHeld)

```
SELECT (SELECT COUNT(*)
          FROM pets p
         WHERE p.shelter_id = :shelterId
           AND p.status IN ('AVAILABLE','PENDING'))
     + (SELECT COUNT(*)
          FROM boarding_requests b
         WHERE b.shelter_id = :shelterId
           AND b.status IN ('ACCEPTED','ACTIVE')
           AND b.start_date <= :end
           AND b.end_date   >= :start)
```

file name: `boarding/BoardingRequestRepository.java`
api name: POST /api/v1/boarding
POST /api/v1/daycare/subscriptions
POST /api/v1/staff/boarding/{id}/accept
Query: (JPQL · SELECT · lockOverlappingForPet)

```
SELECT b FROM BoardingRequest b
 WHERE b.petId = :petId
   AND b.status IN (com.pawhome.backend.boarding.BoardingStatus.ACCEPTED,
                    com.pawhome.backend.boarding.BoardingStatus.ACTIVE)
   AND b.startDate <= :end
   AND b.endDate   >= :start
 ORDER BY b.startDate
```

file name: `boarding/BoardingRequestRepository.java`
api name: POST /api/v1/staff/boarding/{id}/accept
POST /api/v1/staff/boarding/{id}/suite
POST /api/v1/staff/daycare/attendance
Query: (JPQL · SELECT · lockOverlappingForSuite)

```
SELECT b FROM BoardingRequest b
 WHERE b.suiteId = :suiteId
   AND b.status IN (com.pawhome.backend.boarding.BoardingStatus.ACCEPTED,
                    com.pawhome.backend.boarding.BoardingStatus.ACTIVE)
   AND b.startDate <= :end
   AND b.endDate   >= :start
```

file name: `boarding/BoardingRequestRepository.java`
api name: GET /api/v1/staff/boarding/{id}/suites
Query: (JPQL · SELECT · suiteLoads)

```
SELECT     su.id AS suiteId,
           su.capacity AS capacity,
           ((SELECT COUNT(*)
               FROM boarding_requests b
              WHERE b.suite_id = su.id
                AND b.status IN ('ACCEPTED','ACTIVE')
                AND b.start_date <= :end
                AND b.end_date   >= :start)
          + (SELECT COUNT(*)
               FROM daycare_attendance a
              WHERE a.suite_id        = su.id
                AND a.visit_date      = CURDATE()
                AND a.checked_out_at IS NULL
                AND CURDATE() BETWEEN :start AND :end)) AS placesUsed
FROM       shelter_suites su
WHERE      su.shelter_id = :shelterId
```

boarding/BoardingService.java — 7 queries

file name: `boarding/BoardingService.java`
api name: GET /api/v1/boarding/{id}
GET /api/v1/staff/boarding/{id}
POST /api/v1/staff/boarding/{id}/charges
POST /api/v1/staff/boarding/{id}/complete
Query: (SQL · SELECT · chargeRows)

```
SELECT  c.id, c.description, c.quantity, c.unit_price,
        c.line_total, c.added_at,
        -- LEFT, because added_by is nullable: fk_charges_user
        -- is ON DELETE SET NULL so a staff member who has
        -- since left does not take the charge with them.
        u.full_name AS added_by_name
FROM        boarding_charges c
LEFT JOIN   users u ON u.id = c.added_by
WHERE   c.boarding_id = ?
ORDER BY c.added_at ASC, c.id ASC
```

file name: `boarding/BoardingService.java`
api name: GET /api/v1/staff/boarding/counts
Query: (SQL · SELECT · counts)

```
SELECT SUM(b.status = 'REQUESTED')                       AS requested,
       SUM(b.status = 'ACTIVE')                          AS active,
       SUM(b.status = 'ACTIVE' AND b.end_date = CURDATE()) AS departing,
       SUM(b.status = 'COMPLETED')                       AS completed
  FROM boarding_requests b
 WHERE (? IS NULL OR b.shelter_id = ?)
```

file name: `boarding/BoardingService.java`
api name: GET /api/v1/boarding/{id}
GET /api/v1/staff/boarding/{id}
Query: (SQL · SELECT · detail)

```
SELECT u.id, u.update_type, u.note, u.photo_path, u.posted_at,
       a.full_name AS posted_by_name
  FROM boarding_updates u
  JOIN users a ON a.id = u.posted_by
 WHERE u.boarding_id = ? ORDER BY u.posted_at DESC, u.id DESC
```

file name: `boarding/BoardingService.java`
api name: GET /api/v1/staff/boarding
Query: (SQL · SELECT · forShelter)

```
SELECT  b.id, b.pet_id, p.name AS pet_name, p.breed AS pet_breed,
        p.species AS pet_species,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        b.owner_id, o.full_name AS owner_name, o.email AS owner_email,
        b.shelter_id, s.name AS shelter_name, s.address_line AS shelter_address,
        s.phone AS shelter_phone,
        b.suite_id, su.name AS suite_name,
        b.start_date, b.end_date, b.picked_up_on, b.status,
        b.emergency_contact, b.vet_care_authorised,
        b.daily_rate, b.estimated_cost, b.requested_at,
        r.full_name AS reviewed_by_name, b.reviewed_at, b.decline_message,
        q.score AS quiz_score, q.max_score AS quiz_max_score, q.passed AS quiz_passed
FROM        boarding_requests b
JOIN        pets           p  ON p.id  = b.pet_id
JOIN        users          o  ON o.id  = b.owner_id
JOIN        shelters       s  ON s.id  = b.shelter_id
LEFT JOIN   shelter_suites su ON su.id = b.suite_id
LEFT JOIN   users          r  ON r.id  = b.reviewed_by
-- LEFT, because quiz_attempt_id is nullable: V6 notes a request may
-- predate the quiz, and an inner join would hide those rows entirely.
LEFT JOIN   quiz_attempts  q  ON q.id  = b.quiz_attempt_id
WHERE (? IS NULL OR b.shelter_id = ?)
ORDER BY FIELD(b.status,'REQUESTED','ACTIVE','ACCEPTED','COMPLETED',
                        'DECLINED','CANCELLED'),
         b.start_date
```

file name: `boarding/BoardingService.java`
api name: GET /api/v1/boarding/mine
Query: (SQL · SELECT · mine)

```
SELECT  b.id, b.pet_id, p.name AS pet_name, p.breed AS pet_breed,
        p.species AS pet_species,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        b.owner_id, o.full_name AS owner_name, o.email AS owner_email,
        b.shelter_id, s.name AS shelter_name, s.address_line AS shelter_address,
        s.phone AS shelter_phone,
        b.suite_id, su.name AS suite_name,
        b.start_date, b.end_date, b.picked_up_on, b.status,
        b.emergency_contact, b.vet_care_authorised,
        b.daily_rate, b.estimated_cost, b.requested_at,
        r.full_name AS reviewed_by_name, b.reviewed_at, b.decline_message,
        q.score AS quiz_score, q.max_score AS quiz_max_score, q.passed AS quiz_passed
FROM        boarding_requests b
JOIN        pets           p  ON p.id  = b.pet_id
JOIN        users          o  ON o.id  = b.owner_id
JOIN        shelters       s  ON s.id  = b.shelter_id
LEFT JOIN   shelter_suites su ON su.id = b.suite_id
LEFT JOIN   users          r  ON r.id  = b.reviewed_by
-- LEFT, because quiz_attempt_id is nullable: V6 notes a request may
-- predate the quiz, and an inner join would hide those rows entirely.
LEFT JOIN   quiz_attempts  q  ON q.id  = b.quiz_attempt_id
 WHERE b.owner_id = ? ORDER BY b.start_date DESC
```

file name: `boarding/BoardingService.java`
api name: GET /api/v1/boarding/{id}
GET /api/v1/staff/boarding/{id}
POST /api/v1/boarding
POST /api/v1/boarding/{id}/cancel
POST /api/v1/staff/boarding/{id}/accept
POST /api/v1/staff/boarding/{id}/check-in
POST /api/v1/staff/boarding/{id}/complete
POST /api/v1/staff/boarding/{id}/decline
POST /api/v1/staff/boarding/{id}/suite
Query: (SQL · SELECT · row)

```
SELECT  b.id, b.pet_id, p.name AS pet_name, p.breed AS pet_breed,
        p.species AS pet_species,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        b.owner_id, o.full_name AS owner_name, o.email AS owner_email,
        b.shelter_id, s.name AS shelter_name, s.address_line AS shelter_address,
        s.phone AS shelter_phone,
        b.suite_id, su.name AS suite_name,
        b.start_date, b.end_date, b.picked_up_on, b.status,
        b.emergency_contact, b.vet_care_authorised,
        b.daily_rate, b.estimated_cost, b.requested_at,
        r.full_name AS reviewed_by_name, b.reviewed_at, b.decline_message,
        q.score AS quiz_score, q.max_score AS quiz_max_score, q.passed AS quiz_passed
FROM        boarding_requests b
JOIN        pets           p  ON p.id  = b.pet_id
JOIN        users          o  ON o.id  = b.owner_id
JOIN        shelters       s  ON s.id  = b.shelter_id
LEFT JOIN   shelter_suites su ON su.id = b.suite_id
LEFT JOIN   users          r  ON r.id  = b.reviewed_by
-- LEFT, because quiz_attempt_id is nullable: V6 notes a request may
-- predate the quiz, and an inner join would hide those rows entirely.
LEFT JOIN   quiz_attempts  q  ON q.id  = b.quiz_attempt_id
 WHERE b.id = ?
```

file name: `boarding/BoardingService.java`
api name: POST /api/v1/staff/boarding/{id}/updates
Query: (SQL · SELECT · updateRow)

```
SELECT u.id, u.update_type, u.note, u.photo_path, u.posted_at,
       a.full_name AS posted_by_name
  FROM boarding_updates u
  JOIN users a ON a.id = u.posted_by
 WHERE u.id = ?
```

boarding/BoardingUpdateRepository.java — 1 query

file name: `boarding/BoardingUpdateRepository.java`
api name: (not called by anything — see the dead-query note)
Query: (JPQL · SELECT · findByBoardingIdOrderByPostedAtDescIdDesc)

```
SELECT *
  FROM boarding_updates
 WHERE boarding_id = :boardingId
 ORDER BY posted_at DESC, id DESC
```

care/CareDueJob.java — 2 queries

file name: `care/CareDueJob.java`
api name: Scheduled job — CareDueJob.announceDueTasks (@Scheduled)
Query: **(SQL · SELECT · announceDueTasks)**

```
SELECT  t.id, t.description, t.priority, t.assigned_to,
        p.name       AS pet_name,
        p.shelter_id AS shelter_id
  FROM  care_tasks t
  JOIN  pets p ON p.id = t.pet_id
 WHERE  t.status          = 'PENDING'
   AND  t.due_at          < NOW()
   -- V12: the guard that makes this run once per deadline
   -- rather than once per sweep, for ever.
   AND  t.due_notified_at IS NULL
```

file name: `care/CareDueJob.java`
api name: Scheduled job — CareDueJob.announceDueTasks (@Scheduled)
Query: **(SQL · UPDATE · announceDueTasks)**

```
UPDATE care_tasks SET due_notified_at = NOW() WHERE id = ?
```

care/CareService.java — 6 queries

file name: `care/CareService.java`
api name: GET /api/v1/staff/care/colleagues
Query: **(SQL · SELECT · colleagues)**

```
SELECT id, full_name FROM users
 WHERE role = 'STAFF' AND is_active = TRUE AND approval_status = 'APPROVED'
   AND (? IS NULL OR shelter_id = ?)
 ORDER BY full_name
```

file name: `care/CareService.java`
api name: DELETE /api/v1/staff/care/tasks/{id}
GET /api/v1/staff/care
POST /api/v1/staff/care/logs
POST /api/v1/staff/care/tasks
POST /api/v1/staff/care/tasks/{id}/complete
POST /api/v1/staff/care/tasks/{id}/skip
PUT /api/v1/staff/care/tasks/{id}
Query: **(SQL · SELECT · counts)**

```
SELECT SUM(t.status = 'PENDING')                              AS pending,
       SUM(t.status = 'PENDING' AND t.due_at < NOW())        AS overdue,
       SUM(t.status = 'PENDING' AND t.priority = 'CRITICAL') AS critical,
       SUM(t.status = 'DONE'
           AND DATE(t.completed_at) = CURDATE())             AS done_today
  FROM care_tasks t
  JOIN pets p ON p.id = t.pet_id
 WHERE (? IS NULL OR p.shelter_id = ?)
```

file name: `care/CareService.java`
api name: DELETE /api/v1/staff/care/tasks/{id}
GET /api/v1/staff/care
POST /api/v1/staff/care/logs
POST /api/v1/staff/care/tasks
POST /api/v1/staff/care/tasks/{id}/complete
POST /api/v1/staff/care/tasks/{id}/skip
PUT /api/v1/staff/care/tasks/{id}
Query: **(SQL · SELECT · logs)**

```
SELECT  l.id, l.pet_id, p.name AS pet_name, l.task_id, l.task_type,
        l.notes, l.logged_at, u.full_name AS logged_by_name
FROM        care_logs l
JOIN        pets  p ON p.id = l.pet_id
JOIN        users u ON u.id = l.logged_by
WHERE (? IS NULL OR p.shelter_id = ?)
ORDER BY l.logged_at DESC, l.id DESC
```

file name: `care/CareService.java`
api name: POST /api/v1/staff/care/logs
POST /api/v1/staff/care/tasks
Query: (SQL · SELECT · openBoardingFor)

```
SELECT id FROM boarding_requests
 WHERE pet_id = ? AND status = 'ACTIVE'
 ORDER BY start_date DESC LIMIT 1
```

file name: `care/CareService.java`
api name: POST /api/v1/staff/care/tasks
PUT /api/v1/staff/care/tasks/{id}
Query: (SQL · SELECT · requireColleague)

```
SELECT COUNT(*) FROM users
 WHERE id = ? AND role = 'STAFF' AND shelter_id = ?
   AND is_active = TRUE AND approval_status = 'APPROVED'
```

file name: `care/CareService.java`
api name: DELETE /api/v1/staff/care/tasks/{id}
GET /api/v1/staff/care
POST /api/v1/staff/care/logs
POST /api/v1/staff/care/tasks
POST /api/v1/staff/care/tasks/{id}/complete
POST /api/v1/staff/care/tasks/{id}/skip
PUT /api/v1/staff/care/tasks/{id}
Query: (SQL · SELECT · tasks)

```
SELECT  t.id, t.pet_id, p.name AS pet_name,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        t.boarding_id, t.task_type, t.description, t.due_at, t.priority, t.status,
        t.assigned_to, a.full_name AS assigned_to_name,
        c.full_name AS completed_by_name, t.completed_at,
        (t.status = 'PENDING' AND t.due_at < NOW()) AS overdue
FROM        care_tasks t
JOIN        pets  p ON p.id = t.pet_id
LEFT JOIN   users a ON a.id = t.assigned_to
LEFT JOIN   users c ON c.id = t.completed_by
WHERE (? IS NULL OR p.shelter_id = ?)
ORDER BY FIELD(t.status,'PENDING','SKIPPED','DONE'),
         t.priority = 'CRITICAL' DESC,
         t.due_at
```

daycare/DaycareAttendanceRepository.java — 3 queries

file name: `daycare/DaycareAttendanceRepository.java`
api name: POST /api/v1/staff/daycare/attendance/{id}/check-out
Query: (JPQL · UPDATE · checkOut)

```
UPDATE daycare_attendance
   SET checked_out_at = :at,
       checked_out_by = :staffId
 WHERE id             = :id
   AND checked_out_at IS NULL
```

file name: `daycare/DaycareAttendanceRepository.java`
api name: POST /api/v1/staff/daycare/attendance
Query: (JPQL · SELECT · countPresentInSuiteBand)

```
SELECT COUNT(*)
  FROM daycare_attendance
 WHERE suite_id        = :suiteId
   AND visit_date      = :onDate
   AND slot            = :#{#slot.name()}
   AND checked_out_at IS NULL
```

file name: `daycare/DaycareAttendanceRepository.java`
api name: POST /api/v1/staff/daycare/attendance
Query: **(JPQL · SELECT · findVisit)**

```
SELECT *
  FROM daycare_attendance
 WHERE subscription_id = :subscriptionId
   AND visit_date      = :onDate
```

daycare/DaycareService.java — 11 queries

file name: `daycare/DaycareService.java`
api name: POST /api/v1/staff/daycare/attendance
POST /api/v1/staff/daycare/attendance/{id}/check-out
Query: **(SQL · SELECT · attendanceRow)**

```
SELECT  v.attendance_id      AS id,
        v.subscription_id, v.slot, v.visit_date, v.suite_id,
        v.checked_in_at, v.checked_out_at,
        v.suite_name, v.pet_id, v.pet_name, v.owner_name,
        v.checked_in_by_name, v.checked_out_by_name,
        v.minutes_on_site
FROM        v_daycare_day v
 WHERE v.attendance_id = ?
```

file name: `daycare/DaycareService.java`
api name: GET /api/v1/staff/daycare/attendance
Query: **(SQL · SELECT · board)**

```
SELECT  v.attendance_id      AS id,
        v.subscription_id, v.slot, v.visit_date, v.suite_id,
        v.checked_in_at, v.checked_out_at,
        v.suite_name, v.pet_id, v.pet_name, v.owner_name,
        v.checked_in_by_name, v.checked_out_by_name,
        v.minutes_on_site
FROM        v_daycare_day v
WHERE v.shelter_id = ?
  AND v.visit_date = ?
ORDER BY v.still_here DESC,
         FIELD(v.slot,'MORNING','MIDDAY','AFTERNOON'),
         v.checked_in_at
```

file name: `daycare/DaycareService.java`
api name: GET /api/v1/staff/daycare/today
Query: **(SQL · SELECT · dueToday)**

```
SELECT  d.id AS subscription_id, d.slot,
        p.id AS pet_id, p.name AS pet_name, p.species,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE
          LIMIT 1)                      AS pet_cover_photo_path,
        o.full_name AS owner_name, o.email AS owner_email,
        a.id            AS visit_id,
        a.checked_in_at,
        a.checked_out_at,
        su.name         AS suite_name
FROM        daycare_subscriptions d
JOIN        pets  p ON p.id = d.pet_id
JOIN        users o ON o.id = d.owner_id
LEFT JOIN   daycare_attendance a
        ON  a.subscription_id = d.id
        AND a.visit_date      = CURDATE()
LEFT JOIN   shelter_suites su ON su.id = a.suite_id
WHERE   d.shelter_id = ?
  AND   d.status     = 'ACTIVE'
  AND   d.start_date <= CURDATE()
  AND  (d.end_date IS NULL OR d.end_date >= CURDATE())
ORDER BY FIELD(d.slot,'MORNING','MIDDAY','AFTERNOON'),
         a.id IS NOT NULL,
         p.name
```

file name: `daycare/DaycareService.java`
api name: GET /api/v1/admin/daycare/subscriptions
GET /api/v1/staff/daycare/subscriptions
Query: **(SQL · SELECT · forShelter)**

```
SELECT  d.id, d.pet_id, p.name AS pet_name, p.breed AS pet_breed,
        p.species AS pet_species,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        d.owner_id, o.full_name AS owner_name, o.email AS owner_email,
        d.shelter_id, s.name AS shelter_name, s.address_line AS shelter_address,
        s.phone AS shelter_phone,
        d.slot, d.monthly_price, d.start_date, d.end_date, d.status, d.created_at,
        q.score AS quiz_score, q.max_score AS quiz_max_score, q.passed AS quiz_passed,
        -- Correlated, not a join. A join to billing_slips would return
        -- one row per slip and multiply the subscription itself.
        (SELECT COUNT(*) FROM billing_slips bs
          WHERE bs.subscription_id = d.id) AS slips_issued,
        -- TODAY, as three correlated reads rather than a join: a join
        -- to daycare_attendance would be fine here (uq_visit_per_day
        -- means at most one row matches) but it would stop being fine
        -- the moment somebody widened the date filter, and then a
        -- subscription would appear once per visit.
        (SELECT v.id FROM daycare_attendance v
          WHERE v.subscription_id = d.id AND v.visit_date = CURDATE()
            AND v.checked_out_at IS NULL)                 AS today_visit_id,
        (SELECT su.name FROM daycare_attendance v
           JOIN shelter_suites su ON su.id = v.suite_id
          WHERE v.subscription_id = d.id AND v.visit_date = CURDATE()
            AND v.checked_out_at IS NULL)                 AS today_suite_name
FROM        daycare_subscriptions d
JOIN        pets          p ON p.id = d.pet_id
JOIN        users         o ON o.id = d.owner_id
JOIN        shelters      s ON s.id = d.shelter_id
-- INNER, unlike the boarding equivalent: quiz_attempt_id is NOT NULL
-- here, so there is no ungated subscription for a LEFT JOIN to rescue.
JOIN        quiz_attempts q ON q.id = d.quiz_attempt_id
WHERE d.shelter_id = ?
ORDER BY d.status = 'ACTIVE' DESC,
         FIELD(d.slot,'MORNING','MIDDAY','AFTERNOON'),
         d.start_date DESC
```

file name: `daycare/DaycareService.java`
api name: GET /api/v1/staff/daycare/today
Query: **(SQL · SELECT · housingPlaces)**

```
SELECT COALESCE(SUM(su.capacity), 0)
  FROM shelter_suites su
 WHERE su.shelter_id = ?
   AND su.is_active
```

file name: daycare/DaycareService.java
api name: GET /api/v1/daycare/subscriptions/mine
Query: (SQL · SELECT · mine)

```
SELECT  d.id, d.pet_id, p.name AS pet_name, p.breed AS pet_breed,
        p.species AS pet_species,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        d.owner_id, o.full_name AS owner_name, o.email AS owner_email,
        d.shelter_id, s.name AS shelter_name, s.address_line AS shelter_address,
        s.phone AS shelter_phone,
        d.slot, d.monthly_price, d.start_date, d.end_date, d.status, d.created_at,
        q.score AS quiz_score, q.max_score AS quiz_max_score, q.passed AS quiz_passed,
        -- Correlated, not a join. A join to billing_slips would return
        -- one row per slip and multiply the subscription itself.
        (SELECT COUNT(*) FROM billing_slips bs
          WHERE bs.subscription_id = d.id) AS slips_issued,
        -- TODAY, as three correlated reads rather than a join: a join
        -- to daycare_attendance would be fine here (uq_visit_per_day
        -- means at most one row matches) but it would stop being fine
        -- the moment somebody widened the date filter, and then a
        -- subscription would appear once per visit.
        (SELECT v.id FROM daycare_attendance v
          WHERE v.subscription_id = d.id AND v.visit_date = CURDATE()
            AND v.checked_out_at IS NULL)                  AS today_visit_id,
        (SELECT su.name FROM daycare_attendance v
           JOIN shelter_suites su ON su.id = v.suite_id
          WHERE v.subscription_id = d.id AND v.visit_date = CURDATE()
            AND v.checked_out_at IS NULL)                  AS today_suite_name
FROM        daycare_subscriptions d
JOIN        pets          p ON p.id = d.pet_id
JOIN        users         o ON o.id = d.owner_id
JOIN        shelters      s ON s.id = d.shelter_id
-- INNER, unlike the boarding equivalent: quiz_attempt_id is NOT NULL
-- here, so there is no ungated subscription for a LEFT JOIN to rescue.
JOIN        quiz_attempts q ON q.id = d.quiz_attempt_id
WHERE d.owner_id = ?
ORDER BY d.status = 'ACTIVE' DESC, d.start_date DESC, d.id DESC
```

file name: daycare/DaycareService.java
api name: GET /api/v1/daycare/offers
Query: (SQL · SELECT · offers)

```
SELECT id, name, city, state, daycare_monthly_price, daycare_slot_capacity
  FROM shelters
 WHERE status = 'ACTIVE'
   -- A shelter that has never set a price is not running
   -- daycare. The capacity column defaults to 20 for every
   -- shelter, so capacity alone would list all of them with a
   -- free subscription button.
   AND daycare_monthly_price > 0
   AND daycare_slot_capacity > 0
 ORDER BY name
```

file name: daycare/DaycareService.java
api name: POST /api/v1/daycare/subscriptions
Query: (SQL · SELECT · requireRoomFor)

```
SELECT p.species, COUNT(*) AS demand
  FROM daycare_subscriptions d
  JOIN pets p ON p.id = d.pet_id
 WHERE d.shelter_id = ?
   AND d.slot       = ?
   AND d.status     = 'ACTIVE'
 GROUP BY p.species
```

file name: daycare/DaycareService.java
api name: POST /api/v1/daycare/subscriptions
Query: (SQL · SELECT · requireRoomFor)

```
SELECT su.species, SUM(su.capacity) AS places
  FROM shelter_suites su
 WHERE su.shelter_id = ?
   AND su.is_active
 GROUP BY su.species
```

file name: `daycare/DaycareService.java`
api name: GET /api/v1/daycare/subscriptions/{id}
POST /api/v1/daycare/subscriptions
POST /api/v1/daycare/subscriptions/{id}/cancel
Query: (SQL · SELECT · row)

```
SELECT  d.id, d.pet_id, p.name AS pet_name, p.breed AS pet_breed,
        p.species AS pet_species,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE LIMIT 1) AS pet_cover_photo_path,
        d.owner_id, o.full_name AS owner_name, o.email AS owner_email,
        d.shelter_id, s.name AS shelter_name, s.address_line AS shelter_address,
        s.phone AS shelter_phone,
        d.slot, d.monthly_price, d.start_date, d.end_date, d.status, d.created_at,
        q.score AS quiz_score, q.max_score AS quiz_max_score, q.passed AS quiz_passed,
        -- Correlated, not a join. A join to billing_slips would return
        -- one row per slip and multiply the subscription itself.
        (SELECT COUNT(*) FROM billing_slips bs
          WHERE bs.subscription_id = d.id) AS slips_issued,
        -- TODAY, as three correlated reads rather than a join: a join
        -- to daycare_attendance would be fine here (uq_visit_per_day
        -- means at most one row matches) but it would stop being fine
        -- the moment somebody widened the date filter, and then a
        -- subscription would appear once per visit.
        (SELECT v.id FROM daycare_attendance v
          WHERE v.subscription_id = d.id AND v.visit_date = CURDATE()
            AND v.checked_out_at IS NULL)                  AS today_visit_id,
        (SELECT su.name FROM daycare_attendance v
           JOIN shelter_suites su ON su.id = v.suite_id
          WHERE v.subscription_id = d.id AND v.visit_date = CURDATE()
            AND v.checked_out_at IS NULL)                  AS today_suite_name
FROM        daycare_subscriptions d
JOIN        pets          p ON p.id = d.pet_id
JOIN        users         o ON o.id = d.owner_id
JOIN        shelters      s ON s.id = d.shelter_id
-- INNER, unlike the boarding equivalent: quiz_attempt_id is NOT NULL
-- here, so there is no ungated subscription for a LEFT JOIN to rescue.
JOIN        quiz_attempts q ON q.id = d.quiz_attempt_id
 WHERE d.id = ?
```

file name: `daycare/DaycareService.java`
api name: GET /api/v1/daycare/offers
GET /api/v1/staff/daycare/today
Query: (SQL · SELECT · takenBySlot)

```
SELECT shelter_id, slot, COUNT(*) AS taken
  FROM daycare_subscriptions
 WHERE status = 'ACTIVE'
   AND (? IS NULL OR shelter_id = ?)
 GROUP BY shelter_id, slot
```

daycare/DaycareSubscriptionRepository.java — 8 queries

file name: `daycare/DaycareSubscriptionRepository.java`
api name: Scheduled job — MonthlyBillingJob.billTheMonth -> DaycareService.retireEnded
Query: (JPQL · UPDATE · expireEndedBefore)

```
UPDATE daycare_subscriptions
   SET status = 'EXPIRED'
 WHERE status = 'ACTIVE'
   AND end_date IS NOT NULL
   AND end_date < :onDate
```

file name: `daycare/DaycareSubscriptionRepository.java`
api name: Scheduled job — MonthlyBillingJob.billTheMonth (@Scheduled)
Query: (JPQL · SELECT · findActiveOnOrBefore)

```
SELECT *
  FROM daycare_subscriptions
 WHERE status      = 'ACTIVE'
   AND start_date <= :periodTo
 ORDER BY id
```

file name: `daycare/DaycareSubscriptionRepository.java`
api name: (not called by anything — see the dead-query note)
Query: **(JPQL · SELECT · findForOwner)**

```
SELECT *
  FROM daycare_subscriptions
 WHERE owner_id = :ownerId
 ORDER BY status = 'ACTIVE' DESC, start_date DESC, id DESC
```

file name: `daycare/DaycareSubscriptionRepository.java`
api name: (not called by anything — see the dead-query note)
Query: **(JPQL · SELECT · findForShelter)**

```
SELECT *
  FROM daycare_subscriptions
 WHERE shelter_id = :shelterId
 ORDER BY status = 'ACTIVE' DESC,
          FIELD(slot, 'MORNING', 'MIDDAY', 'AFTERNOON'),
          start_date DESC
```

file name: `daycare/DaycareSubscriptionRepository.java`
api name: POST /api/v1/daycare/subscriptions
Query: **(JPQL · SELECT · lockPetSlot)**

```
SELECT s FROM DaycareSubscription s
 WHERE s.petId  = :petId
   AND s.slot   = :slot
   AND s.status = com.pawhome.backend.daycare.SubscriptionStatus.ACTIVE
```

file name: `daycare/DaycareSubscriptionRepository.java`
api name: POST /api/v1/boarding
POST /api/v1/staff/boarding/{id}/accept
Query: **(JPQL · SELECT · lockPetSubscriptionsOver)**

```
SELECT s FROM DaycareSubscription s
 WHERE s.petId  = :petId
   AND s.status = com.pawhome.backend.daycare.SubscriptionStatus.ACTIVE
   AND s.startDate <= :end
   AND (s.endDate IS NULL OR s.endDate >= :start)
 ORDER BY s.id
```

file name: `daycare/DaycareSubscriptionRepository.java`
api name: POST /api/v1/daycare/subscriptions
Query: **(JPQL · SELECT · lockSlotAtShelter)**

```
SELECT s FROM DaycareSubscription s
 WHERE s.shelterId = :shelterId
   AND s.slot      = :slot
   AND s.status    = com.pawhome.backend.daycare.SubscriptionStatus.ACTIVE
 ORDER BY s.id
```

file name: `daycare/DaycareSubscriptionRepository.java`
api name: POST /api/v1/daycare/subscriptions/{id}/cancel
Query: **(JPQL · UPDATE · stop)**

```
UPDATE daycare_subscriptions
   SET status   = :#{#status.name()},
       end_date = :endDate
 WHERE id       = :id
   AND status   = 'ACTIVE'
```

health/VaccinationService.java — 4 queries

file name: `health/VaccinationService.java`
api name: DELETE /api/v1/my-pets/{id}/vaccinations/{vaccinationId}
DELETE /api/v1/staff/pets/{petId}/vaccinations/{id}
GET /api/v1/pets/{petId}/health
GET /api/v1/staff/pets/{petId}/vaccinations
POST /api/v1/my-pets/{id}/vaccinations
POST /api/v1/my-pets/{id}/vaccinations/{vaccinationId}/certificate
POST /api/v1/staff/pets/{petId}/vaccinations
POST /api/v1/staff/pets/{petId}/vaccinations/{id}/certificate
PUT /api/v1/staff/pets/{petId}/vaccinations/{id}
Query: (SQL · SELECT · forPet)

```
SELECT id FROM vaccinations WHERE pet_id = ? AND recorded_by = ?
```

file name: `health/VaccinationService.java`
api name: GET /api/v1/staff/vaccinations
Query: (SQL · SELECT · forShelter)

```
SELECT  v.id, v.pet_id, p.name AS pet_name,
        v.vaccine_type_id, vt.name AS vaccine_name,
        v.administered_on, v.next_due_on, v.administered_by, v.batch_lot,
        v.status, v.certificate_path, v.notes,
        r.full_name AS recorded_by_name,
        (v.status = 'SCHEDULED' AND v.next_due_on < CURDATE()) AS overdue
FROM        vaccinations  v
JOIN        pets          p  ON p.id  = v.pet_id
JOIN        vaccine_types vt ON vt.id = v.vaccine_type_id
JOIN        users         r  ON r.id  = v.recorded_by
WHERE p.shelter_id IS NOT NULL AND (? IS NULL OR p.shelter_id = ?)
ORDER BY v.status DESC, v.next_due_on IS NULL, v.next_due_on, v.id DESC
```

file name: `health/VaccinationService.java`
api name: DELETE /api/v1/my-pets/{id}/vaccinations/{vaccinationId}
DELETE /api/v1/staff/pets/{petId}/vaccinations/{id}
GET /api/v1/pets/{petId}/health
GET /api/v1/staff/pets/{petId}/vaccinations
POST /api/v1/my-pets/{id}/vaccinations
POST /api/v1/my-pets/{id}/vaccinations/{vaccinationId}/certificate
POST /api/v1/staff/pets/{petId}/vaccinations
POST /api/v1/staff/pets/{petId}/vaccinations/{id}/certificate
PUT /api/v1/staff/pets/{petId}/vaccinations/{id}
Query: (SQL · SELECT · rows)

```
SELECT  v.id, v.pet_id, p.name AS pet_name,
        v.vaccine_type_id, vt.name AS vaccine_name,
        v.administered_on, v.next_due_on, v.administered_by, v.batch_lot,
        v.status, v.certificate_path, v.notes,
        r.full_name AS recorded_by_name,
        (v.status = 'SCHEDULED' AND v.next_due_on < CURDATE()) AS overdue
FROM        vaccinations  v
JOIN        pets          p  ON p.id  = v.pet_id
JOIN        vaccine_types vt ON vt.id = v.vaccine_type_id
JOIN        users         r  ON r.id  = v.recorded_by
 WHERE v.pet_id = ? ORDER BY v.administered_on DESC, v.id DESC
```

file name: `health/VaccinationService.java`
api name: DELETE /api/v1/my-pets/{id}/vaccinations/{vaccinationId}
DELETE /api/v1/staff/pets/{petId}/vaccinations/{id}
GET /api/v1/pets/{petId}/health
GET /api/v1/staff/pets/{petId}/vaccinations
POST /api/v1/my-pets/{id}/vaccinations
POST /api/v1/my-pets/{id}/vaccinations/{vaccinationId}/certificate
POST /api/v1/staff/pets/{petId}/vaccinations
POST /api/v1/staff/pets/{petId}/vaccinations/{id}/certificate
PUT /api/v1/staff/pets/{petId}/vaccinations/{id}
Query: **(SQL · SELECT · vaccinated)**

```
SELECT (    EXISTS (SELECT 1 FROM vaccinations v
                    WHERE v.pet_id = ? AND v.status = 'COMPLETED')
        AND NOT EXISTS (SELECT 1 FROM vaccinations v2
                        WHERE v2.pet_id = ?
                          AND v2.status = 'SCHEDULED'
                          AND v2.next_due_on < CURDATE()))
```

health/VaccineTypeRepository.java — 1 query

file name: `health/VaccineTypeRepository.java`
api name: GET /api/v1/my-pets/{id}/vaccinations/types
GET /api/v1/staff/pets/{petId}/vaccinations/types
Query: **(JPQL · SELECT · findByActiveTrueOrderByNameAsc)**

```
SELECT *
  FROM vaccine_types
 WHERE is_active = TRUE
 ORDER BY name ASC
```

mail/HealthReminderJob.java — 2 queries

file name: `mail/HealthReminderJob.java`
api name: Scheduled job — HealthReminderJob.sendDueReminders (@Scheduled)
Query: **(SQL · INSERT · run)**

```
INSERT IGNORE INTO health_reminders_sent
    (vaccination_id, reminder_kind, sent_to)
VALUES (?, ?, ?)
```

file name: `mail/HealthReminderJob.java`

api name: Scheduled job — HealthReminderJob.sendDueReminders (@Scheduled)

Query: (SQL · SELECT · run)

```
SELECT  v.id            AS vaccination_id,
        p.name          AS pet_name,
        u.full_name     AS owner_name,
        u.email         AS owner_email,
        vt.name         AS vaccine_name,
        v.next_due_on   AS due_on,
        s.name          AS shelter_name
FROM        vaccinations  v
JOIN        pets          p  ON p.id  = v.pet_id
-- INNER JOIN through owner_id: a pet with no owner is still in the
-- shelter, and there is nobody to email about it.
JOIN        users         u  ON u.id  = p.owner_id
JOIN        vaccine_types vt ON vt.id = v.vaccine_type_id
LEFT JOIN   shelters      s  ON s.id  = p.shelter_id
WHERE   v.next_due_on IS NOT NULL
  AND   v.status = 'SCHEDULED'
  AND   %s
  -- Only people who confirmed their address and are still active.
  AND   u.email_verified = TRUE
  AND   u.is_active      = TRUE
  -- V10's switches, and this is the one that makes them real.
  -- notify_health is the topic; notify_email is the channel, so
  -- turning either off stops this mail -- and the settings page
  -- stops being a row of switches wired to nothing.
  AND   u.notify_health  = TRUE
  AND   u.notify_email   = TRUE
  -- The idempotence guard: skip anything already sent.
  AND   NOT EXISTS (SELECT 1 FROM health_reminders_sent r
                    WHERE r.vaccination_id = v.id
                      AND r.reminder_kind  = ?)
```

notification/NotificationRepository.java — 3 queries

file name: `notification/NotificationRepository.java`

api name: GET /api/v1/notifications

POST /api/v1/admin/users/{id}/approve

POST /api/v1/admin/users/{id}/reject

POST /api/v1/applications

POST /api/v1/notifications/read

POST /api/v1/notifications/{id}/read

POST /api/v1/staff/applications/{id}/approve

POST /api/v1/staff/applications/{id}/decline

POST /api/v1/staff/boarding/{id}/accept

POST /api/v1/staff/boarding/{id}/check-in

POST /api/v1/staff/boarding/{id}/complete

POST /api/v1/staff/boarding/{id}/decline

POST /api/v1/staff/boarding/{id}/suite

POST /api/v1/staff/boarding/{id}/updates

Query: (JPQL · SELECT · countByUserIdAndReadAtIsNull)

```
SELECT COUNT(*)
  FROM notifications
 WHERE user_id = :userId
   AND read_at IS NULL
```

file name: `notification/NotificationRepository.java`

api name: GET /api/v1/notifications

POST /api/v1/notifications/read

POST /api/v1/notifications/{id}/read

Query: (JPQL · SELECT · findRecentForUser)

```
SELECT *
  FROM notifications
 WHERE user_id = :userId
 ORDER BY created_at DESC, id DESC
 LIMIT :limit
```

file name: `notification/NotificationRepository.java`
api name: POST /api/v1/notifications/read
Query: (JPQL · UPDATE · markAllRead)

```
UPDATE Notification n SET n.readAt = :now
```

notification/NotificationService.java — 1 query

file name: `notification/NotificationService.java`
api name: POST /api/v1/applications
Query: (SQL · SELECT · raiseForShelterStaff)

```
SELECT id FROM users
 WHERE role = 'STAFF' AND shelter_id = ?
   AND is_active = TRUE AND approval_status = 'APPROVED'
```

pet/FavoriteService.java — 3 queries

file name: `pet/FavoriteService.java`
api name: PUT /api/v1/favorites/{petId}
Query: (SQL · SELECT · add)

```
SELECT EXISTS (SELECT 1 FROM pets WHERE id = ?)
```

file name: `pet/FavoriteService.java`
api name: PUT /api/v1/favorites/{petId}
Query: (SQL · INSERT · add)

```
INSERT IGNORE INTO favorites (user_id, pet_id) VALUES (?, ?)
```

file name: `pet/FavoriteService.java`
api name: DELETE /api/v1/favorites/{petId}
Query: (SQL · DELETE · remove)

```
DELETE FROM favorites WHERE user_id = ? AND pet_id = ?
```

pet/PetPhotoRepository.java — 3 queries

file name: `pet/PetPhotoRepository.java`
api name: POST /api/v1/my-pets/{id}/photos
POST /api/v1/staff/pets/{id}/photos
Query: (JPQL · SELECT · countByPetId)

```
SELECT COUNT(*)
  FROM pet_photos
 WHERE pet_id = :petId
```

file name: `pet/PetPhotoRepository.java`
api name: POST /api/v1/my-pets/{id}/photos
POST /api/v1/my-pets/{id}/photos/{photoId}/cover
POST /api/v1/staff/pets/{id}/photos
POST /api/v1/staff/pets/{id}/photos/{photoId}/cover
Query: (JPQL · SELECT · findByPetIdAndCoverTrue)

```
SELECT *
  FROM pet_photos
 WHERE pet_id = :petId
   AND is_cover = TRUE
```

file name: `pet/PetPhotoRepository.java`
api name: DELETE /api/v1/my-pets/{id}
DELETE /api/v1/my-pets/{id}/photos/{photoId}
DELETE /api/v1/staff/pets/{id}
DELETE /api/v1/staff/pets/{id}/photos/{photoId}
GET /api/v1/my-pets/{id}
GET /api/v1/staff/pets/{id}
POST /api/v1/my-pets/{id}/photos
POST /api/v1/my-pets/{id}/photos/{photoId}/cover
POST /api/v1/staff/pets
POST /api/v1/staff/pets/{id}/photos
POST /api/v1/staff/pets/{id}/photos/{photoId}/cover
PUT /api/v1/staff/pets/{id}
Query: **(JPQL · SELECT · findByPetIdOrderBySortOrderAscIdAsc)**

```
SELECT *
  FROM pet_photos
 WHERE pet_id = :petId
 ORDER BY sort_order ASC, id ASC
```

pet/PetRepository.java — 2 queries

file name: `pet/PetRepository.java`
api name: (not called by anything — see the dead-query note)
Query: **(JPQL · SELECT · countByShelterId)**

```
SELECT COUNT(*)
  FROM pets
 WHERE shelter_id = :shelterId
```

file name: `pet/PetRepository.java`
api name: POST /api/v1/staff/applications/{id}/approve
Query: **(JPQL · UPDATE · markAdopted)**

```
UPDATE pets
   SET status   = 'ADOPTED',
       owner_id = :ownerId
 WHERE id       = :petId
   AND status  IN ('AVAILABLE', 'PENDING')
```

pet/PetService.java — 14 queries

file name: `pet/PetService.java`
api name: GET /api/v1/pets
Query: (SQL · SELECT · browse)

```
SELECT  vl.pet_id      AS id,
        vl.pet_name    AS name,
        vl.species, vl.breed, vl.age_value, vl.age_unit, vl.sex,
        vl.size, vl.weight_kg, vl.status, vl.description,
        vl.is_neutered, vl.is_microchipped, vl.adoption_fee,
        vl.shelter_name, vl.shelter_city, vl.shelter_state,
        vl.cover_photo_path,
        vl.photo_paths,
        -- The '||' spelling, not the human-readable ', ' one. The view
        -- carries both because two readers want two things.
        vl.trait_label_list AS trait_labels,
        -- Saved by the viewer. ONE placeholder, and it comes before
        -- every WHERE parameter, so callers pass viewerId first.
        --
        -- `<=>` is MySQL's null-safe equality. It is NOT load-bearing
        -- here: with a NULL viewer id, plain `=` yields NULL for every
        -- row, the subquery returns none, and EXISTS is FALSE -- the
        -- right answer. It stays because it says "a null viewer matches
        -- nobody" in the operator instead of leaving it to three-valued
        -- logic to collapse the right way.
        EXISTS (SELECT 1 FROM favorites f
                 WHERE f.pet_id = vl.pet_id AND f.user_id <=> ?) AS favorite
FROM       v_pet_listing vl
WHERE vl.status = 'AVAILABLE'
  AND (? IS NULL OR vl.species = ?)
ORDER BY vl.created_at DESC
```

file name: `pet/PetService.java`
api name: GET /api/v1/staff/pets/counts
Query: (SQL · SELECT · counts)

```
SELECT  COUNT(*)                     AS total,
        SUM(status = 'AVAILABLE')    AS available,
        SUM(status = 'PENDING')      AS pending,
        SUM(status = 'ADOPTED')      AS adopted,
        SUM(status = 'BOARDING')     AS boarding,
        SUM(status = 'UNAVAILABLE')  AS unavailable
FROM pets WHERE (? IS NULL OR shelter_id = ?)
```

file name: `pet/PetService.java`
api name: GET /api/v1/favorites
Query: (SQL · SELECT · favorites)

```
SELECT  vl.pet_id      AS id,
        vl.pet_name    AS name,
        vl.species, vl.breed, vl.age_value, vl.age_unit, vl.sex,
        vl.size, vl.weight_kg, vl.status, vl.description,
        vl.is_neutered, vl.is_microchipped, vl.adoption_fee,
        vl.shelter_name, vl.shelter_city, vl.shelter_state,
        vl.cover_photo_path,
        vl.photo_paths,
        -- The '||' spelling, not the human-readable ', ' one. The view
        -- carries both because two readers want two things.
        vl.trait_label_list AS trait_labels,
        -- Saved by the viewer. ONE placeholder, and it comes before
        -- every WHERE parameter, so callers pass viewerId first.
        --
        -- `<=>` is MySQL's null-safe equality. It is NOT load-bearing
        -- here: with a NULL viewer id, plain `=` yields NULL for every
        -- row, the subquery returns none, and EXISTS is FALSE -- the
        -- right answer. It stays because it says "a null viewer matches
        -- nobody" in the operator instead of leaving it to three-valued
        -- logic to collapse the right way.
        EXISTS (SELECT 1 FROM favorites f
                 WHERE f.pet_id = vl.pet_id AND f.user_id <=> ?) AS favorite
FROM       v_pet_listing vl
JOIN favorites fav ON fav.pet_id = vl.pet_id AND fav.user_id = ?
ORDER BY fav.created_at DESC
```

file name: `pet/PetService.java`
api name: GET /api/v1/staff/pets
Query: (SQL · SELECT · list)

```
SELECT  p.id, p.shelter_id, s.name AS shelter_name, p.name, p.species,
        p.breed, p.age_value, p.age_unit, p.sex, p.size, p.weight_kg,
        p.status, p.owner_id, p.description, p.is_neutered,
        p.is_microchipped, p.adoption_fee, p.view_count, p.created_at,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE
          LIMIT 1)                                        AS cover_photo_path,
        (SELECT COUNT(*) FROM pet_photos ph
          WHERE ph.pet_id = p.id)                         AS photo_count,
        (SELECT GROUP_CONCAT(pt.trait_id ORDER BY pt.trait_id)
           FROM pet_traits pt WHERE pt.pet_id = p.id)     AS trait_ids,
        (SELECT GROUP_CONCAT(t.label ORDER BY t.label SEPARATOR '||')
           FROM pet_traits pt JOIN traits t ON t.id = pt.trait_id
          WHERE pt.pet_id = p.id)                         AS trait_labels,
        (SELECT COUNT(*) FROM adoption_applications a
          WHERE a.pet_id = p.id)                          AS application_count,
        -- The three tables that reference a pet ON DELETE RESTRICT,
        -- as one number. Every other table pointing at a pet cascades:
        -- photos, traits, favourites, care logs, care tasks and
        -- vaccinations all belong to the animal and go with it.
        ((SELECT COUNT(*) FROM adoption_applications a WHERE a.pet_id = p.id)
       + (SELECT COUNT(*) FROM adoptions ad           WHERE ad.pet_id = p.id)
       + (SELECT COUNT(*) FROM boarding_requests b    WHERE b.pet_id = p.id))
                                                        AS blocking_count
FROM       pets     p
-- INNER, and load-bearing since V9. shelter_id is nullable now and a
-- NULL means a household pet, so this join is what keeps somebody's own
-- dog out of a shelter's Manage Pets list. Change it to a LEFT JOIN and
-- every staff member starts seeing every customer's animals.
JOIN       shelters s ON s.id = p.shelter_id
WHERE (? IS NULL OR p.shelter_id = ?)
ORDER BY p.created_at DESC
```

file name: `pet/PetService.java`
api name: DELETE /api/v1/my-pets/{id}/photos/{photoId}
GET /api/v1/my-pets/{id}
POST /api/v1/my-pets
POST /api/v1/my-pets/{id}/photos
POST /api/v1/my-pets/{id}/photos/{photoId}/cover
PUT /api/v1/my-pets/{id}
Query: (SQL · SELECT · myPet)

```
SELECT  p.id, p.name, p.species, p.breed, p.age_value, p.age_unit,
        p.sex, p.size, p.weight_kg, p.status, p.description,
        p.is_neutered, p.is_microchipped,
        p.shelter_id, s.name AS shelter_name,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE
          LIMIT 1)                                        AS cover_photo_path
FROM       pets     p
LEFT JOIN  shelters s ON s.id = p.shelter_id
 WHERE p.id = ?
```

file name: `pet/PetService.java`
api name: GET /api/v1/my-pets
Query: (SQL · SELECT · myPets)

```
SELECT  p.id, p.name, p.species, p.breed, p.age_value, p.age_unit,
        p.sex, p.size, p.weight_kg, p.status, p.description,
        p.is_neutered, p.is_microchipped,
        p.shelter_id, s.name AS shelter_name,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE
          LIMIT 1)                                        AS cover_photo_path
FROM       pets     p
LEFT JOIN  shelters s ON s.id = p.shelter_id
 WHERE p.owner_id = ? ORDER BY p.name
```

file name: `pet/PetService.java`
api name: GET /api/v1/pets/{id}
Query: **(SQL · SELECT · publicPet)**

```
SELECT  vl.pet_id      AS id,
        vl.pet_name    AS name,
        vl.species, vl.breed, vl.age_value, vl.age_unit, vl.sex,
        vl.size, vl.weight_kg, vl.status, vl.description,
        vl.is_neutered, vl.is_microchipped, vl.adoption_fee,
        vl.shelter_name, vl.shelter_city, vl.shelter_state,
        vl.cover_photo_path,
        vl.photo_paths,
        -- The '||' spelling, not the human-readable ', ' one. The view
        -- carries both because two readers want two things.
        vl.trait_label_list AS trait_labels,
        -- Saved by the viewer. ONE placeholder, and it comes before
        -- every WHERE parameter, so callers pass viewerId first.
        --
        -- `<=>` is MySQL's null-safe equality. It is NOT load-bearing
        -- here: with a NULL viewer id, plain `=` yields NULL for every
        -- row, the subquery returns none, and EXISTS is FALSE -- the
        -- right answer. It stays because it says "a null viewer matches
        -- nobody" in the operator instead of leaving it to three-valued
        -- logic to collapse the right way.
        EXISTS (SELECT 1 FROM favorites f
                 WHERE f.pet_id = vl.pet_id AND f.user_id <=> ?) AS favorite
FROM       v_pet_listing vl
 WHERE vl.pet_id = ?
```

file name: `pet/PetService.java`
api name: DELETE /api/v1/my-pets/{id}
Query: **(SQL · SELECT · removeMyPet)**

```
SELECT COUNT(*) FROM boarding_requests WHERE pet_id = ?
```

file name: `pet/PetService.java`
api name: DELETE /api/v1/staff/pets/{id}
DELETE /api/v1/staff/pets/{id}/photos/{photoId}
GET /api/v1/staff/pets/{id}
POST /api/v1/staff/pets
POST /api/v1/staff/pets/{id}/photos
POST /api/v1/staff/pets/{id}/photos/{photoId}/cover
POST /api/v1/staff/pets/{id}/status
PUT /api/v1/staff/pets/{id}
Query: (SQL · SELECT · row)

```
SELECT  p.id, p.shelter_id, s.name AS shelter_name, p.name, p.species,
        p.breed, p.age_value, p.age_unit, p.sex, p.size, p.weight_kg,
        p.status, p.owner_id, p.description, p.is_neutered,
        p.is_microchipped, p.adoption_fee, p.view_count, p.created_at,
        (SELECT ph.file_path FROM pet_photos ph
          WHERE ph.pet_id = p.id AND ph.is_cover = TRUE
          LIMIT 1)                                      AS cover_photo_path,
        (SELECT COUNT(*) FROM pet_photos ph
          WHERE ph.pet_id = p.id)                       AS photo_count,
        (SELECT GROUP_CONCAT(pt.trait_id ORDER BY pt.trait_id)
           FROM pet_traits pt WHERE pt.pet_id = p.id)    AS trait_ids,
        (SELECT GROUP_CONCAT(t.label ORDER BY t.label SEPARATOR '||')
           FROM pet_traits pt JOIN traits t ON t.id = pt.trait_id
          WHERE pt.pet_id = p.id)                       AS trait_labels,
        (SELECT COUNT(*) FROM adoption_applications a
          WHERE a.pet_id = p.id)                        AS application_count,
        -- The three tables that reference a pet ON DELETE RESTRICT,
        -- as one number. Every other table pointing at a pet cascades:
        -- photos, traits, favourites, care logs, care tasks and
        -- vaccinations all belong to the animal and go with it.
        ((SELECT COUNT(*) FROM adoption_applications a WHERE a.pet_id = p.id)
       + (SELECT COUNT(*) FROM adoptions ad           WHERE ad.pet_id = p.id)
       + (SELECT COUNT(*) FROM boarding_requests b    WHERE b.pet_id = p.id))
                                                       AS blocking_count
FROM       pets     p
-- INNER, and load-bearing since V9. shelter_id is nullable now and a
-- NULL means a household pet, so this join is what keeps somebody's own
-- dog out of a shelter's Manage Pets list. Change it to a LEFT JOIN and
-- every staff member starts seeing every customer's animals.
JOIN       shelters s ON s.id = p.shelter_id
 WHERE p.id = ?
```

file name: `pet/PetService.java`
api name: GET /api/v1/favorites
GET /api/v1/pets
GET /api/v1/pets/{id}
Query: (SQL · SELECT · withMatch)

```
SELECT trait_label FROM v_adopter_traits WHERE user_id = ? ORDER BY trait_label
```

file name: `pet/PetService.java`
api name: GET /api/v1/favorites
GET /api/v1/pets
GET /api/v1/pets/{id}
Query: (SQL · SELECT · withMatch)

```
SELECT pt.pet_id,
       GROUP_CONCAT(va.trait_label ORDER BY va.trait_label SEPARATOR '||') AS shared
  FROM v_adopter_traits va
  JOIN pet_traits pt ON pt.trait_id = va.trait_id
 WHERE va.user_id = ? AND pt.pet_id IN (%s)
 GROUP BY pt.pet_id
```

file name: `pet/PetService.java`
api name: POST /api/v1/staff/pets
PUT /api/v1/staff/pets/{id}
Query: (SQL · DELETE · writeTraits)

```
DELETE FROM pet_traits WHERE pet_id = ?
```

file name: `pet/PetService.java`
api name: POST /api/v1/staff/pets
PUT /api/v1/staff/pets/{id}
Query: (SQL · SELECT · writeTraits)

```
SELECT id FROM traits WHERE is_active = TRUE
```

file name: `pet/PetService.java`
api name: POST /api/v1/staff/pets
PUT /api/v1/staff/pets/{id}
Query: (SQL · INSERT · writeTraits)

```
INSERT INTO pet_traits (pet_id, trait_id) VALUES (?, ?)
```

quiz/QuizAttemptRepository.java — 1 query

file name: `quiz/QuizAttemptRepository.java`
api name: GET /api/v1/quiz/{quizType}/attempts/latest
Query: (JPQL · SELECT · findFirstByUserIdAndQuizTypeOrderByIdDesc)

```
SELECT *
  FROM quiz_attempts
 WHERE user_id = :userId
   AND quiz_type = :#{#quizType.name()}
 ORDER BY id DESC
 LIMIT 1
```

quiz/QuizOptionRepository.java — 3 queries

file name: `quiz/QuizOptionRepository.java`
api name: (not called by anything — see the dead-query note)
Query: (JPQL · DELETE · deleteByQuestionId)

```
DELETE FROM QuizOption o WHERE o.questionId = :questionId
```

file name: `quiz/QuizOptionRepository.java`
api name: GET /api/v1/quiz/{quizType}/questions
POST /api/v1/quiz/{quizType}/attempts
Query: (JPQL · SELECT · findByQuestionIdInOrderByDisplayOrderAsc)

```
SELECT *
  FROM quiz_options
 WHERE question_id IN (:questionIds)
 ORDER BY display_order ASC
```

file name: `quiz/QuizOptionRepository.java`
api name: GET /api/v1/admin/quiz/questions/{id}
POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (JPQL · SELECT · findByQuestionIdOrderByDisplayOrderAsc)

```
SELECT *
  FROM quiz_options
 WHERE question_id = :questionId
 ORDER BY display_order ASC
```

quiz/QuizQuestionRepository.java — 3 queries

file name: `quiz/QuizQuestionRepository.java`
api name: (not called by anything — see the dead-query note)
Query: **(JPQL · SELECT · countByQuizType)**

```
SELECT COUNT(*)
  FROM quiz_questions
 WHERE quiz_type = :#{#quizType.name()}
```

file name: `quiz/QuizQuestionRepository.java`
api name: (not called by anything — see the dead-query note)
Query: **(JPQL · SELECT · countByQuizTypeAndStatus)**

```
SELECT COUNT(*)
  FROM quiz_questions
 WHERE quiz_type = :#{#quizType.name()}
   AND status = :#{#status.name()}
```

file name: `quiz/QuizQuestionRepository.java`
api name: GET /api/v1/quiz/{quizType}/questions
POST /api/v1/admin/quiz/questions
POST /api/v1/admin/quiz/questions/reorder
POST /api/v1/quiz/{quizType}/attempts
Query: **(JPQL · SELECT · findByQuizTypeOrderByDisplayOrderAsc)**

```
SELECT *
  FROM quiz_questions
 WHERE quiz_type = :#{#quizType.name()}
 ORDER BY display_order ASC
```

quiz/QuizService.java — 8 queries

file name: `quiz/QuizService.java`
api name: GET /api/v1/admin/quiz/counts
Query: **(SQL · SELECT · counts)**

```
SELECT  COUNT(*)                            AS total,
        SUM(quiz_type = 'ELIGIBILITY')      AS eligibility,
        SUM(quiz_type = 'PERSONALITY')      AS personality,
        SUM(quiz_type = 'BOARDING')         AS boarding,
        SUM(status    = 'ACTIVE')           AS active
FROM quiz_questions
```

file name: `quiz/QuizService.java`
api name: GET /api/v1/admin/quiz/questions
GET /api/v1/admin/quiz/questions/{id}
POST /api/v1/admin/quiz/questions
POST /api/v1/admin/quiz/questions/reorder
POST /api/v1/admin/quiz/questions/{id}/status
PUT /api/v1/admin/quiz/questions/{id}
Query: **(SQL · SELECT · list)**

```
SELECT  q.id, q.quiz_type, q.question_text, q.answer_type, q.category,
        q.status, q.is_required, q.display_order, q.created_at,
        (SELECT COUNT(*) FROM quiz_options o
          WHERE o.question_id = q.id)                   AS option_count,
        (SELECT COUNT(*) FROM quiz_answers a
          WHERE a.question_id = q.id)                   AS answered_count
FROM        quiz_questions q
WHERE       q.quiz_type = ?
ORDER BY    q.display_order, q.id
```

file name: `quiz/QuizService.java`
api name: GET /api/v1/admin/quiz/questions/{id}
POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (SQL · SELECT · options)

```
SELECT ot.option_id, ot.trait_id
  FROM option_traits ot
  JOIN quiz_options o ON o.id = ot.option_id
 WHERE o.question_id = ?
```

file name: `quiz/QuizService.java`
api name: GET /api/v1/admin/quiz/questions/{id}
POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (SQL · SELECT · options)

```
SELECT DISTINCT a.option_id
  FROM quiz_answers a
  JOIN quiz_options o ON o.id = a.option_id
 WHERE o.question_id = ?
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (SQL · UPDATE · writeOptions)

```
UPDATE quiz_options SET display_order = ? WHERE id = ?
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (SQL · DELETE · writeTraits)

```
DELETE FROM option_traits WHERE option_id IN (
)
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (SQL · INSERT · writeTraits)

```
INSERT INTO option_traits (option_id, trait_id) VALUES (?, ?)
```

file name: `quiz/QuizService.java`
api name: POST /api/v1/admin/quiz/questions
PUT /api/v1/admin/quiz/questions/{id}
Query: (SQL · DELETE · writeTraits)

```
DELETE FROM option_traits WHERE option_id IN (
```

quiz/QuizTakingService.java — 1 query

file name: `quiz/QuizTakingService.java`
api name: GET /api/v1/traits/mine
Query: (SQL · SELECT · myTraits)

```
SELECT trait_label FROM v_adopter_traits WHERE user_id = ? ORDER BY trait_label
```

quiz/TraitRepository.java — 1 query

file name: `quiz/TraitRepository.java`
api name: GET /api/v1/traits
Query: (JPQL · SELECT · findByActiveTrueOrderByLabelAsc)

```
SELECT *
  FROM traits
 WHERE is_active = TRUE
 ORDER BY label ASC
```

report/ReportService.java — 12 queries

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · adoptionSummary)

```
SELECT a.adopted_on, p.name AS pet, p.species, p.breed,
       sh.name AS shelter, u.full_name AS adopter, u.email,
       a.fee_paid,
       DATEDIFF(a.adopted_on, DATE(app.submitted_at)) AS days
  FROM adoptions a
  JOIN pets p                   ON p.id   = a.pet_id
  JOIN shelters sh              ON sh.id  = a.shelter_id
  JOIN users u                  ON u.id   = a.adopter_id
  JOIN adoption_applications app ON app.id = a.application_id
 WHERE a.adopted_on BETWEEN ? AND ?
   AND (? IS NULL OR a.shelter_id = ?)
 ORDER BY a.adopted_on, p.name
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · generate)

```
SELECT name FROM shelters WHERE id = ?
```

file name: `report/ReportService.java`
api name: GET /api/v1/admin/reports
POST /api/v1/admin/reports
Query: (SQL · SELECT · list)

```
SELECT r.id, r.file_name, r.file_path, r.report_type,
       r.date_from, r.date_to, r.shelter_id,
       sh.name AS shelter_name, u.full_name AS generated_by_name,
       r.generated_at
  FROM reports r
  LEFT JOIN shelters sh ON sh.id = r.shelter_id
  LEFT JOIN users u     ON u.id  = r.generated_by
 ORDER BY r.generated_at DESC, r.id DESC
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · petInventory)

```
SELECT sh.name AS shelter, p.name AS pet, p.species, p.breed,
       p.sex, p.size, p.age_value, p.age_unit, p.status,
       p.adoption_fee,
       (    EXISTS (SELECT 1 FROM vaccinations v
                    WHERE v.pet_id = p.id AND v.status = 'COMPLETED')
        AND NOT EXISTS (SELECT 1 FROM vaccinations v2
                        WHERE v2.pet_id = p.id
                          AND v2.status = 'SCHEDULED'
                          AND v2.next_due_on < CURDATE())
       )                                          AS vaccinated,
       (SELECT COUNT(*) FROM adoption_applications a2
         WHERE a2.pet_id = p.id AND a2.status = 'PENDING') AS open_apps
  FROM pets p
  JOIN shelters sh ON sh.id = p.shelter_id
 WHERE p.shelter_id IS NOT NULL
   AND (? IS NULL OR p.shelter_id = ?)
 ORDER BY sh.name, p.species, p.name
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · quarterlyImpact)

```
SELECT SUM(a.fee_paid) FROM adoptions a
 WHERE a.adopted_on BETWEEN ? AND ? AND (? IS NULL OR a.shelter_id = ?)
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · quarterlyImpact)

```
SELECT COUNT(*) FROM adoptions a
 WHERE a.adopted_on BETWEEN ? AND ? AND (? IS NULL OR a.shelter_id = ?)
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · quarterlyImpact)

```
SELECT COUNT(*) FROM adoption_applications app JOIN pets p ON p.id = app.pet_id
 WHERE app.submitted_at >= ? AND app.submitted_at < DATE_ADD(?, INTERVAL 1 DAY)
   AND (? IS NULL OR p.shelter_id = ?)
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · quarterlyImpact)

```
SELECT COUNT(*) FROM adoption_applications app JOIN pets p ON p.id = app.pet_id
 WHERE app.status = 'APPROVED'
   AND app.submitted_at >= ? AND app.submitted_at < DATE_ADD(?, INTERVAL 1 DAY)
   AND (? IS NULL OR p.shelter_id = ?)
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · quarterlyImpact)

```
SELECT COUNT(*) FROM adoption_applications app JOIN pets p ON p.id = app.pet_id
 WHERE app.status IN ('APPROVED','REJECTED')
   AND app.submitted_at >= ? AND app.submitted_at < DATE_ADD(?, INTERVAL 1 DAY)
   AND (? IS NULL OR p.shelter_id = ?)
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · quarterlyImpact)

```
SELECT COUNT(*) FROM pets p
 WHERE p.shelter_id IS NOT NULL AND p.status IN ('AVAILABLE','PENDING')
   AND (? IS NULL OR p.shelter_id = ?)
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · quarterlyImpact)

```
SELECT COUNT(*) FROM boarding_requests b
 WHERE b.status = 'COMPLETED' AND b.end_date BETWEEN ? AND ?
   AND (? IS NULL OR b.shelter_id = ?)
```

file name: `report/ReportService.java`
api name: POST /api/v1/admin/reports
Query: (SQL · SELECT · shelterEfficiency)

```
SELECT sh.name AS shelter, sh.city, sh.capacity,
       (SELECT COUNT(*) FROM pets p2
         WHERE p2.shelter_id = sh.id)                    AS pets_held,
       COUNT(app.id)                                     AS applications,
       SUM(app.status = 'APPROVED')                      AS approved,
       SUM(app.status = 'REJECTED')                      AS rejected,
       SUM(app.status = 'PENDING')                       AS pending,
       SUM(app.status = 'WITHDRAWN')                     AS withdrawn,
       ROUND(SUM(app.status = 'APPROVED') * 100
             / NULLIF(SUM(app.status IN ('APPROVED','REJECTED')), 0)
            )                                            AS approval_pct,
       ROUND(AVG(CASE WHEN app.reviewed_at IS NOT NULL
                 THEN TIMESTAMPDIFF(HOUR, app.submitted_at, app.reviewed_at) / 24
                 END), 1)                                AS avg_days,
       (SELECT COUNT(*) FROM adoptions a
         WHERE a.shelter_id = sh.id
           AND a.adopted_on BETWEEN ? AND ?)             AS adoptions
  FROM shelters sh
  -- The applications reach the shelter through its pets, which
  -- is the only route: adoption_applications has no shelter_id
  -- of its own, and copying one there would let the two
  -- disagree the day a pet is transferred.
  LEFT JOIN pets p ON p.shelter_id = sh.id
  LEFT JOIN adoption_applications app
         ON app.pet_id = p.id
        AND app.submitted_at >= ?
        AND app.submitted_at <  DATE_ADD(?, INTERVAL 1 DAY)
 WHERE (? IS NULL OR sh.id = ?)
 GROUP BY sh.id, sh.name, sh.city, sh.capacity
 ORDER BY sh.name
```

shelter/ShelterRepository.java — 4 queries

file name: `shelter/ShelterRepository.java`
api name: (not called by anything — see the dead-query note)
Query: (JPQL · SELECT · findAllByOrderByNameAsc)

```
SELECT *
  FROM shelters
 ORDER BY name ASC
```

file name: `shelter/ShelterRepository.java`
api name: POST /api/v1/admin/shelters
PUT /api/v1/admin/shelters/{id}
Query: (JPQL · SELECT · findByNameIgnoreCase)

```
SELECT *
  FROM shelters
 WHERE UPPER(name) = UPPER(:name)
```

file name: `shelter/ShelterRepository.java`
api name: GET /api/v1/shelters
Query: (JPQL · SELECT · findByStatusOrderByNameAsc)

```
SELECT *
  FROM shelters
 WHERE status = :#{#status.name()}
 ORDER BY name ASC
```

file name: `shelter/ShelterRepository.java`
api name: POST /api/v1/staff/boarding/{id}/accept
Query: (JPQL · SELECT · lockById)

```
SELECT s FROM Shelter s WHERE s.id = :id
```

shelter/ShelterService.java — 6 queries

file name: `shelter/ShelterService.java`
api name: GET /api/v1/admin/shelters
Query: **(SQL · SELECT · list)**

```
SELECT  s.*,
        (SELECT COUNT(*) FROM pets p
          WHERE p.shelter_id = s.id
            AND p.status <> 'ADOPTED')            AS pet_count,
        (SELECT COUNT(*) FROM users u
          WHERE u.shelter_id = s.id
            AND u.role = 'STAFF'
            AND u.is_active = TRUE)               AS staff_count,
        (SELECT COUNT(*) FROM shelter_suites sc
          WHERE sc.shelter_id = s.id)             AS suite_count,
        -- PLACES, NOT ROOMS. suite_count is how many rooms exist;
        -- this is how many animals they hold between them, which
        -- is the number daycare_slot_capacity has to be set
        -- against. Active rooms only: a retired one is not
        -- somewhere anybody can be put.
        (SELECT COALESCE(SUM(sp.capacity), 0) FROM shelter_suites sp
          WHERE sp.shelter_id = s.id
            AND sp.is_active)                     AS suite_places,
        -- ANIMALS ACTUALLY IN THE BUILDING. Three sources, and
        -- they do not overlap: a resident is one of ours awaiting
        -- a home, a boarder is somebody's pet that has checked in,
        -- a daycare pet is here for one band today.
        --
        -- This is NOT pet_count above. That one is the shelter's
        -- books; both were rendered under the label "Pets in Care",
        -- which is why accepting a boarding request appeared to
        -- change nothing -- a boarded pet has no shelter_id and was
        -- never on those books.
        ((SELECT COUNT(*) FROM pets p2
           WHERE p2.shelter_id = s.id
             AND p2.status IN ('AVAILABLE','PENDING'))
       + (SELECT COUNT(*) FROM boarding_requests b2
           WHERE b2.shelter_id = s.id
             AND b2.status = 'ACTIVE')
       + (SELECT COUNT(*) FROM daycare_attendance a2
           JOIN daycare_subscriptions ds2 ON ds2.id = a2.subscription_id
           WHERE ds2.shelter_id     = s.id
             AND a2.visit_date      = CURDATE()
             AND a2.checked_out_at IS NULL)) AS on_site_count
FROM      shelters s
ORDER BY  s.name
```

file name: `shelter/ShelterService.java`
api name: GET /api/v1/shelters/mine/rooms
Query: (SQL · SELECT · myRooms)

```
SELECT  su.id, su.name, su.species, su.is_active, su.capacity,
        b.status       AS boarding_status,
        b.start_date, b.end_date,
        DATEDIFF(CURDATE(), b.start_date) AS nights_so_far,
        bp.id          AS b_pet_id,
        bp.name        AS b_pet_name,
        bo.full_name   AS b_owner_name,
        a.id           AS visit_id,
        a.slot         AS visit_slot,
        dp.id          AS d_pet_id,
        dp.name        AS d_pet_name,
        downer.full_name AS d_owner_name,

        -- HOW FULL THE ROOM IS. The two kinds of occupant are
        -- counted separately and added, because they live in
        -- different tables and there is no row anywhere that
        -- represents "an animal in a room" in general.
        --
        -- These are the SAME two counts BoardingService.accept
        -- and DaycareService.checkIn take under a lock before
        -- admitting an animal. This one is unlocked on purpose
        -- -- it draws a screen, and a room that fills between
        -- the render and the click is caught where it matters.
        ((SELECT COUNT(*)
            FROM boarding_requests b2
           WHERE b2.suite_id = su.id
             AND b2.status IN ('ACCEPTED','ACTIVE')
             AND CURDATE() BETWEEN b2.start_date AND b2.end_date)
       + (SELECT COUNT(*)
            FROM daycare_attendance a2
           WHERE a2.suite_id        = su.id
             AND a2.visit_date      = CURDATE()
             AND a2.checked_out_at IS NULL)) AS occupied

FROM        shelter_suites su

-- THE FACE OF THE ROOM, NOT ALL OF IT. Pinned to one id by a
-- correlated subquery rather than joined on a predicate: a
-- predicate join would return one row per occupant and
-- duplicate the room, which is exactly what stopped being
-- impossible when V16 made capacity a number.
--
-- Earliest arrival wins the line, on the same reasoning as
-- whoIsHere ordering longest-first: the animal that has been
-- here longest is the one somebody needs to think about. Ties
-- break on id so the answer is stable between refreshes.
--
-- ACCEPTED and ACTIVE are exactly the two statuses
-- BoardingStatus.holdsDates() names.
LEFT JOIN   boarding_requests b
        ON  b.id = (SELECT b3.id
                      FROM boarding_requests b3
                     WHERE b3.suite_id = su.id
                       AND b3.status IN ('ACCEPTED','ACTIVE')
                       AND CURDATE() BETWEEN b3.start_date
                                         AND b3.end_date
                     ORDER BY b3.start_date, b3.id
                     LIMIT 1)
LEFT JOIN   pets  bp ON bp.id = b.pet_id
LEFT JOIN   users bo ON bo.id = b.owner_id

-- The day occupant, chosen the same way: here today, not yet
-- gone home, earliest through the door.
LEFT JOIN   daycare_attendance a
        ON  a.id = (SELECT a3.id
                      FROM daycare_attendance a3
                     WHERE a3.suite_id        = su.id
                       AND a3.visit_date      = CURDATE()
                       AND a3.checked_out_at IS NULL
                     ORDER BY a3.checked_in_at, a3.id
                     LIMIT 1)
LEFT JOIN   daycare_subscriptions ds ON ds.id = a.subscription_id
LEFT JOIN   pets  dp     ON dp.id     = ds.pet_id
LEFT JOIN   users downer ON downer.id = ds.owner_id
WHERE   su.shelter_id = ?
ORDER BY su.is_active DESC, su.name
```

file name: `shelter/ShelterService.java`
api name: GET /api/v1/admin/shelters/{id}
GET /api/v1/shelters/mine
POST /api/v1/admin/shelters
POST /api/v1/admin/shelters/{id}/logo
POST /api/v1/admin/shelters/{id}/status
POST /api/v1/admin/users/{id}/shelter
PUT /api/v1/admin/shelters/{id}
Query: (SQL · SELECT · one)

```
SELECT  s.*,
        (SELECT COUNT(*) FROM pets p
          WHERE p.shelter_id = s.id
            AND p.status <> 'ADOPTED')            AS pet_count,
        (SELECT COUNT(*) FROM users u
          WHERE u.shelter_id = s.id
            AND u.role = 'STAFF'
            AND u.is_active = TRUE)               AS staff_count,
        (SELECT COUNT(*) FROM shelter_suites sc
          WHERE sc.shelter_id = s.id)             AS suite_count,
        -- PLACES, NOT ROOMS. suite_count is how many rooms exist;
        -- this is how many animals they hold between them, which
        -- is the number daycare_slot_capacity has to be set
        -- against. Active rooms only: a retired one is not
        -- somewhere anybody can be put.
        (SELECT COALESCE(SUM(sp.capacity), 0) FROM shelter_suites sp
          WHERE sp.shelter_id = s.id
            AND sp.is_active)                     AS suite_places,
        -- ANIMALS ACTUALLY IN THE BUILDING. Three sources, and
        -- they do not overlap: a resident is one of ours awaiting
        -- a home, a boarder is somebody's pet that has checked in,
        -- a daycare pet is here for one band today.
        --
        -- This is NOT pet_count above. That one is the shelter's
        -- books; both were rendered under the label "Pets in Care",
        -- which is why accepting a boarding request appeared to
        -- change nothing -- a boarded pet has no shelter_id and was
        -- never on those books.
        ((SELECT COUNT(*) FROM pets p2
           WHERE p2.shelter_id = s.id
             AND p2.status IN ('AVAILABLE','PENDING'))
       + (SELECT COUNT(*) FROM boarding_requests b2
           WHERE b2.shelter_id = s.id
             AND b2.status = 'ACTIVE')
       + (SELECT COUNT(*) FROM daycare_attendance a2
           JOIN daycare_subscriptions ds2 ON ds2.id = a2.subscription_id
           WHERE ds2.shelter_id     = s.id
             AND a2.visit_date      = CURDATE()
             AND a2.checked_out_at IS NULL)) AS on_site_count
FROM   shelters s
WHERE  s.id = ?
```

file name: `shelter/ShelterService.java`
api name: POST /api/v1/admin/shelters/{id}/status
Query: (SQL · SELECT · setStatus)

```
SELECT COUNT(*) FROM users
 WHERE shelter_id = ? AND role = 'STAFF' AND is_active = TRUE
```

file name: `shelter/ShelterService.java`
api name: POST /api/v1/admin/shelters/{id}/status
Query: (SQL · SELECT · setStatus)

```
SELECT COUNT(*) FROM pets
 WHERE shelter_id = ? AND status <> 'ADOPTED'
```

file name: `shelter/ShelterService.java`
api name: GET /api/v1/shelters/mine/on-site
Query: (SQL · SELECT · whoIsHere)

```
SELECT 'RESIDENT' AS kind, p.id AS pet_id, p.name AS pet_name,
       p.species, NULL AS owner_name, NULL AS room_name,
       DATE(p.created_at) AS since,
       DATEDIFF(CURDATE(), DATE(p.created_at)) AS days_here
  FROM pets p
 WHERE p.shelter_id = ?
   AND p.status IN ('AVAILABLE','PENDING')

UNION ALL

SELECT 'BOARDING', p.id, p.name, p.species, o.full_name, su.name,
       b.start_date,
       DATEDIFF(CURDATE(), b.start_date)
  FROM boarding_requests b
  JOIN pets  p ON p.id = b.pet_id
  JOIN users o ON o.id = b.owner_id
  LEFT JOIN shelter_suites su ON su.id = b.suite_id
 WHERE b.shelter_id = ?
   AND b.status = 'ACTIVE'

UNION ALL

SELECT 'DAYCARE', p.id, p.name, p.species, o.full_name, su.name,
       a.visit_date, 0
  FROM daycare_attendance a
  JOIN daycare_subscriptions ds ON ds.id = a.subscription_id
  JOIN pets  p  ON p.id  = ds.pet_id
  JOIN users o  ON o.id  = ds.owner_id
  JOIN shelter_suites su ON su.id = a.suite_id
 WHERE ds.shelter_id     = ?
   AND a.visit_date      = CURDATE()
   AND a.checked_out_at IS NULL

ORDER BY days_here DESC, pet_name
```

shelter/ShelterSuiteRepository.java — 3 queries

file name: `shelter/ShelterSuiteRepository.java`
api name: (not called by anything — see the dead-query note)
Query: (JPQL · SELECT · countByShelterId)

```
SELECT COUNT(*)
  FROM shelter_suites
 WHERE shelter_id = :shelterId
```

file name: `shelter/ShelterSuiteRepository.java`
api name: POST /api/v1/admin/shelters/{id}/suites
POST /api/v1/shelters/mine/suites
PUT /api/v1/admin/shelters/{id}/suites/{suiteId}
PUT /api/v1/shelters/mine/suites/{suiteId}
Query: (JPQL · SELECT · findByShelterIdAndNameIgnoreCase)

```
SELECT *
  FROM shelter_suites
 WHERE shelter_id = :shelterId
   AND UPPER(name) = UPPER(:name)
```

file name: `shelter/ShelterSuiteRepository.java`
api name: GET /api/v1/admin/shelters/{id}/suites
GET /api/v1/shelters/mine/suites
GET /api/v1/staff/boarding/{id}/suites
Query: (JPQL · SELECT · findByShelterIdOrderByNameAsc)

```
SELECT *
  FROM shelter_suites
 WHERE shelter_id = :shelterId
 ORDER BY name ASC
```

token/UserTokenRepository.java — 3 queries

file name: `token/UserTokenRepository.java`
api name: (not called by anything — see the dead-query note)
Query: (JPQL · DELETE · deleteExpiredBefore)

```
DELETE FROM UserToken t WHERE t.expiresAt < :cutoff
```

file name: `token/UserTokenRepository.java`
api name: POST /api/v1/auth/accept-invite
POST /api/v1/auth/reset-password
POST /api/v1/auth/verify-email
Query: (JPQL · SELECT · findByTokenHash)

```
SELECT *
  FROM user_tokens
 WHERE token_hash = :tokenHash
```

file name: `token/UserTokenRepository.java`
api name: POST /api/v1/admin/users/invite
POST /api/v1/admin/users/{id}/approve
POST /api/v1/admin/users/{id}/resend-invite
POST /api/v1/admin/users/{id}/resend-verification
POST /api/v1/auth/forgot-password
POST /api/v1/auth/register
POST /api/v1/auth/resend-verification
Query: (JPQL · UPDATE · invalidateOutstanding)

```
UPDATE UserToken t SET t.usedAt = :now
```

user/UserRepository.java — 2 queries

file name: `user/UserRepository.java`
api name: POST /api/v1/auth/register
Query: (JPQL · SELECT · existsByEmail)

```
SELECT COUNT(u) > 0 FROM User u WHERE u.email = :email
```

file name: `user/UserRepository.java`
api name: POST /api/v1/admin/users/invite
POST /api/v1/auth/forgot-password
POST /api/v1/auth/login
POST /api/v1/auth/resend-verification
Query: (JPQL · SELECT · findByEmail)

```
SELECT *
  FROM users
 WHERE email = :email
```

Appendix — the queries no endpoint reaches

Found while building this document, and left in rather than quietly dropped. Two different situations, and only the second is a problem.

Run by a scheduled job, not by a request

These are correct. Nothing is wrong with a query that no user triggers — the monthly daycare billing run, the vaccination reminder mail and the care-task announcement are all `@Scheduled` methods, and they are the only callers.

File	Method	Trigger
billing/BillingSlipRepository.java	findForSubscriptionMonth	MonthlyBillingJob.billTheMonth -> BillingService.issueForSubscription
care/CareDueJob.java	announceDueTasks	CareDueJob.announceDueTasks (@Scheduled)
daycare/DaycareSubscriptionRepository.java	expireEndedBefore	MonthlyBillingJob.billTheMonth -> DaycareService.retireEnded
daycare/DaycareSubscriptionRepository.java	findActiveOnOrBefore	MonthlyBillingJob.billTheMonth (@Scheduled)
mail/HealthReminderJob.java	run	HealthReminderJob.sendDueReminders (@Scheduled)

Declared but never called

Each of these is a repository method (or, in one case, a SQL constant) that compiles, would work, and that nothing in the codebase invokes. None of them is a bug — an uncalled query cannot misbehave — but each is a small claim the code makes and does not keep, and for a database course they are worth either wiring up or deleting. Two of them are duplicates of work done another way: `AuditLogRepository.findRecent` and `BoardingChargeRepository.findForBoarding` were each superseded by a JdbcTemplate query in the matching service, and the JPQL version was left behind.

File	Method / constant	Kind
analytics/AnalyticsService.java	APPS_IN_WINDOW	SELECT
application/AdoptionRepository.java	findByApplicationId	SELECT
audit/AuditLogRepository.java	findRecent	SELECT
boarding/BoardingChargeRepository.java	findForBoarding	SELECT
boarding/BoardingUpdateRepository.java	findByBoardingIdOrderByPostedAtDescIdDesc	SELECT
daycare/DaycareSubscriptionRepository.java	findForOwner	SELECT
daycare/DaycareSubscriptionRepository.java	findForShelter	SELECT
pet/PetRepository.java	countByShelterId	SELECT
quiz/QuizOptionRepository.java	deleteByQuestionId	DELETE
quiz/QuizQuestionRepository.java	countByQuizType	SELECT
quiz/QuizQuestionRepository.java	countByQuizTypeAndStatus	SELECT
shelter/ShelterRepository.java	findAllByOrderByNameAsc	SELECT
shelter/ShelterSuiteRepository.java	countByShelterId	SELECT
token/UserTokenRepository.java	deleteExpiredBefore	DELETE

How this document was produced

Not by hand, and not by regular expressions. A scanner walks each of the 171 Java files once, in order, classifying every character as code, comment or string — necessary because this codebase comments heavily and Java text blocks hold SQL full of quotes and braces, which no single regex can tell apart from a brace that closes a method. Members are then split by brace depth, SQL is lifted from text blocks and `@Query` annotations, shared constants are substituted into the statements that concatenate them, and each query is attributed to an endpoint by walking the injected-field call graph from all 151 controller methods.

Checked rather than assumed: an independent grep census counted 60 @Query annotations and 101 JdbcTemplate call sites, and the extractor accounts for every query it found — 145 reached by an endpoint, 7 by a scheduled job, 14 called by nothing, summing to the 166 documented here. Three faults were found and fixed during that check: method names were being read out of the SQL text (three members

came out named `UPPER` *and* `name`*), the call-graph walk was cut off before reaching repositories four calls deep, and shared SQL constants were being listed as queries of their own alongside the statements that include them. Five mappings were then verified by hand against the source.*